

Go Back

Hardware

Nano R4

Tutorials

Nano R4 User Manual

Motor Anomaly Detection with the Nano R4

Nano R4 Bootloader Recovery and Flashing

Working with External Interrupts on Nano R4

Home / Hardware / Nano R4 / Nano R4 User Manual

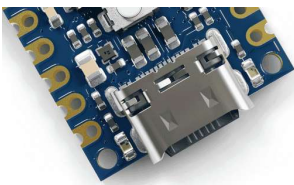
ON THIS PAGE

Nano R4 User Manual

Learn about the hardware and software features of the Arduino® Nano R4 board.

Author · José Bagur Last revision · 10/22/2025

This user manual provides a comprehensive overview of the Nano R4 board, highlighting its hardware and software elements. With it, you will learn how to set up, configure and use all the main features of the Nano R4 board.



ARDUINO
NANO R4
User Manual

Hardware and Software Requirements

Hardware Requirements

- Nano R4 (x1)
- USB-C® cable (x1)
- Breadboard (x1) (recommended)
- Male/male jumper wires (recommended)

Software Requirements

- Arduino IDE or Arduino Cloud Editor
- Arduino UNO R4 Boards core

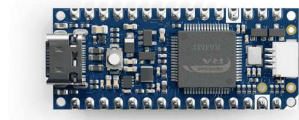
The Nano R4 is compatible with the complete Arduino ecosystem and can be programmed directly as a standalone device.

Hardware and Software Requirements	-
Hardware Requirements	
Software Requirements	
Nano R4 Overview	+
First Use	+
LEDs	+
Pins	+
Digital-to-Analog Converter (DAC)	
Real-Time Clock (RTC)	
EEPROM (Non-Volatile Memory)	
UART Communication	
SPI Communication	
I²C Communication	+
CAN Communication	
HID (Human Interface Device) Communication	
External Interrupts	+
Qwiic Connector	+
Bootloader	+
Support	+

Nano R4 Overview

The Nano R4 board represents the natural evolution of the Nano family, combining the powerful RA4M1 microcontroller from Renesas with the compact and familiar Nano form

from prototyping to production, using the same powerful core already used in the UNO R4 family.

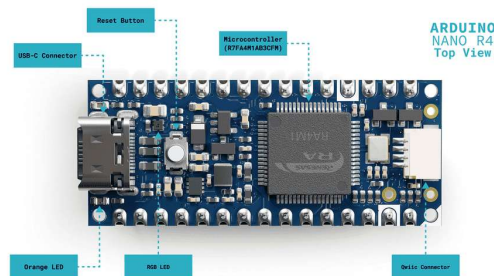


The Nano R4 includes a high-performance 32-bit microcontroller (R7FA4M1AB3CFM), expanded connectivity through an onboard Qwiic connector and advanced features such as DAC, CAN and operational amplifiers. Its compact dimensions (18 mm x 45 mm) and robust construction make the Nano R4 board an excellent choice for projects that demand sensor fusion capabilities and the computational power of modern microcontrollers.

Nano R4 Architecture Overview

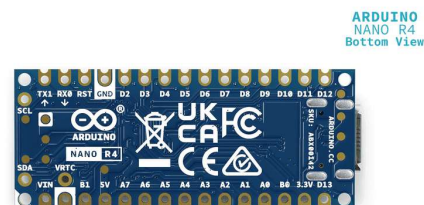
The Nano R4 board features a secure, certified and durable design that suits various applications, such as industrial automation, building automation and rapid prototyping.

The top view of the Nano R4 board is shown in the image below:



The Nano R4 main components (top view)

The bottom view of the Nano R4 board is shown in the image below:



The Nano R4 main components (bottom view)



The bottom side of the Nano R4 board features only test points for debugging and development purposes, along with the board's certification markings and identification information (board model and SKU).

Here is an overview of the board's main components shown in the images above:

Microcontroller: At the heart of the Nano R4 board there is a Renesas RA4M1 family microcontroller ([R7FA4M1AB3CFM](#)). This single-chip microcontroller, recognized as one of the industry's most energy-efficient microcontroller, is based on a 48 MHz Arm® Cortex®-M4 core with up to 256 KB of flash memory and 32 KB of SRAM memory.

USB-C connector: The Nano R4 board features a modern USB-C connector for programming, power supply and serial communication with the external world.

Qwiic connector: The Nano R4 board also includes an onboard Qwiic connector to expand the board's communication capabilities via I²C, facilitating connection with a wide range of boards, sensors, actuators and different peripherals.

Programmable RGB LED: The Nano R4 board has an onboard user-programmable RGB LED to provide visual feedback about different operating states.

User LED: In addition to the onboard user-programmable RGB LED, the board also includes an additional onboard user-programmable orange LED for basic status indications.

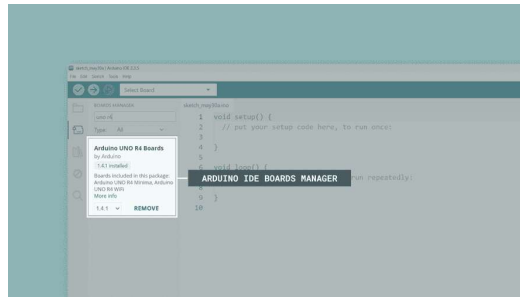
Castellated pins: The board's castellated pins allow surface mounting as a module, facilitating integration into custom hardware designs.

Advanced microcontroller features: The R7FA4M1AB3CFM microcontroller has integrated peripherals such as a 12-bit Digital-to-Analog Converter (DAC), CAN bus for industrial communications, integrated Operational Amplifiers (OpAmp) and HID emulation capabilities (keyboard/mouse).

Board Core and Libraries

The **Arduino UNO R4 Boards** core contains the libraries and examples to work with the Arduino Nano R4's peripherals and onboard components, such as its RA4M1 microcontroller, advanced peripherals (DAC, CAN and OpAmp), Qwiic connector and the onboard RGB LED. To install the core for

Manager or click the **Boards Manager** icon in the left tab of the IDE. In the Boards Manager tab, search for **UNO R4** and install the latest Arduino UNO R4 Boards version.



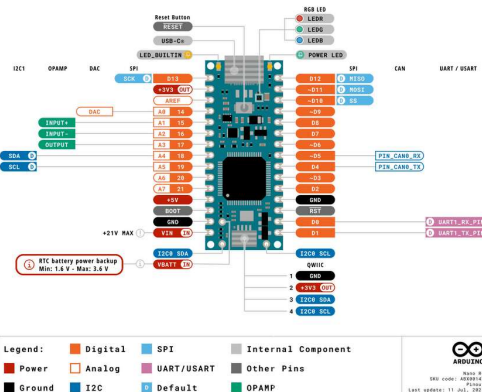
Installing the Arduino UNO R4 Boards core in the Arduino IDE

The Arduino UNO R4 Boards core provides support for the following:

- Board control and configuration (reset, pin configuration and power management)
- Advanced peripheral functions (12-bit DAC, ADC, CAN bus and OpAmp)
- Communication interfaces (UART, I²C and SPI)
- Onboard LED control (RGB LED and orange LED)
- Real-time clock (RTC) functionality
- HID emulation capabilities (keyboard and mouse)
- Standard Arduino libraries compatibility

Important note: Since the Nano R4 uses the same RA4M1 microcontroller as the UNO R4 WiFi and the UNO R4 Minima, it shares complete code and library compatibility, making it easy to transition projects between these boards.

Pinout



Nano R4 pinout

The full pinout is available and downloadable as PDF from

Nano R4 pinout

Datasheet

The complete datasheet is available and downloadable as PDF from the link below:

[Nano R4 datasheet](#)

Schematics

The complete schematics are available and downloadable as PDF from the link below:

[- Nano R4 schematics](#)

STEP Files

The complete STEP files are available and downloadable from the link below:

[Nano R4 STEP files](#)

First Use

Unboxing the Product

When opening the Nano R4 box, you will find the board and its corresponding documentation. **The Nano R4 does not include additional cables**, so you will need a USB-C cable ([available separately here](#)) to connect the board to your computer.



Nano R4 unboxing

The Nano R4 is a standalone device that can be programmed directly without requiring additional boards. However, for more complex projects, you can easily combine it with Arduino shields compatible with the Nano family or connect it to other Arduino devices through its onboard Qwiic connector.

Connecting the Board

The Nano R4 can be connected to your computer using its onboard USB-C connector. It can also be integrated into larger projects using the following:

Direct USB-C connection: For programming, power supply and serial communication with the computer

Pin connection: For integration into breadboards or custom PCBs

Qwiic connection: For rapid expansion with compatible sensors and modules

Module mounting: Using the board's castellated pins for direct soldering to PCBs

 **Important note:** The Nano R4 operates at +5 VDC natively. When connecting sensors or modules that operate at +3.3 VDC, make sure to verify voltage compatibility to avoid component damage.

Voltage Compatibility Considerations

Before connecting any external components to your Nano R4, it is important to understand its voltage characteristics to prevent damage to your sensors and modules:

The Nano R4 operates at +5 VDC, which means:

All digital I/O pins use +5 VDC logic levels (HIGH = +5 VDC, LOW = 0 VDC)

Analog inputs can safely accept 0 to +5 VDC

Communication pins (I²C, SPI, UART) operate at +5 VDC logic levels

For +3.3 VDC components, you have these safe options:

Use the onboard **Qwiic connector** for I²C devices (includes built-in level shifting)

Add **external level shifters** for digital communication pins

Use the onboard **+3.3 VDC power pin** to power your devices (but still need level shifting for data lines)

Common components that typically require level shifting:

Modern sensors from SparkFun (Qwiic), Adafruit (STEMMA QT), and Pimoroni for example

Most MEMS sensors (for example, accelerometers, gyroscopes, pressure sensors)

Many OLED and TFT displays

SD card modules

Most wireless modules (Wi-Fi®, Bluetooth®, LoRa®)

Always check your component's datasheet for voltage specifications before connecting. When in doubt, use a multimeter to verify voltage levels or add protective level shifting.

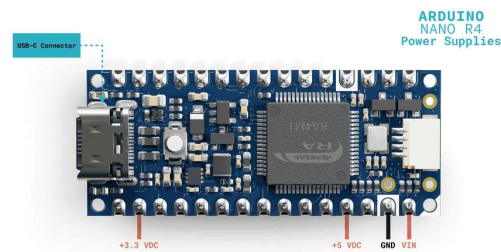
Powering the Board

The Nano R4 can be powered in several ways:

Via USB-C connector: The most common method during development and programming

Via `VIN` pin: Using an external +6-21 VDC power supply that will be internally regulated to +5 VDC

Via `5V` pin: Directly connecting a regulated +5 VDC source (with caution)



Different ways to power the Nano R4 board

Important note: The Nano R4's `VIN` pin accepts a voltage range of +6-21 VDC. Do not connect voltages outside this range as you could permanently damage the board. Always verify all the connections before applying power.

Internal +3.3 VDC Power Supply

The Nano R4 also includes an onboard +3.3 VDC regulator ([AP2112K](#)) that provides power for the following:

Qwiic connector: Supplies +3.3 VDC power to connected I²C devices

I²C level translation: Enables communication between the +5 VDC microcontroller and +3.3 VDC Qwiic devices

Internal +3.3 VDC peripherals: Powers certain internal circuits that require +3.3 VDC operation

This internal +3.3 VDC supply allows the board to interface with both +5 VDC and +3.3 VDC devices through the Qwiic ecosystem while maintaining the +5 VDC operation of the board's main microcontroller.

VBATT Pin

The `VBATT` pin is used to connect a battery to the board.

microcontroller's real-time clock (RTC) and certain low-power functions when the main power source is disconnected from the board. This is particularly useful for applications that need to keep track of time or maintain certain settings of the microcontroller during power outages.

Common battery options for this purpose include the following:

- Coin cell batteries (CR2032: +3 VDC)

- Rechargeable LiPo batteries (+3.7 VDC nominal)

- AA/AAA batteries (+1.5-3 VDC depending on the chemistry of the battery)

Hello World Example

Let's program the Nano R4 to reproduce the classic

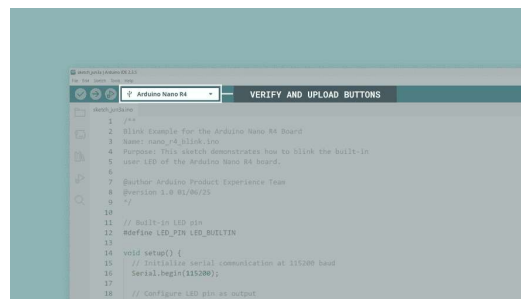
`Hello World` example used in the Arduino ecosystem: the

`Blink` sketch. We will use this example to verify that the Nano R4's connection to the computer works correctly, that the Arduino IDE is properly configured, and that both the board and development environment function as expected.

First, connect your Nano R4 to your computer using a USB-C cable, open the Arduino IDE, and make sure that the board is connected correctly. If you are new to the Arduino IDE, please refer to the official Arduino documentation for more detailed information about initial setup. Copy and paste the following example sketch into a new Arduino IDE file:

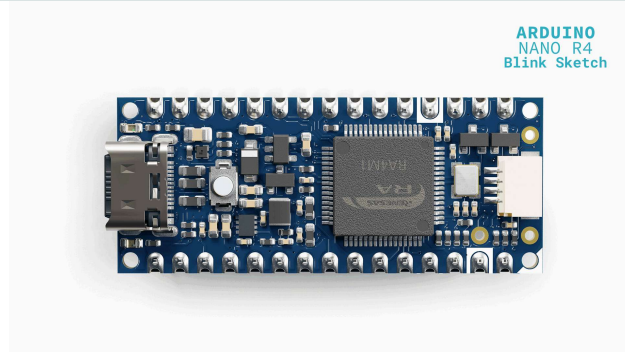
```
1  /**
2  Blink Example for the Arduino Nano R4 Board
3  Name: nano_r4_blink.ino
4  Purpose: This sketch demonstrates how to blink
5  user LED of the Arduino Nano R4 board.
6
7  @author Arduino Product Experience Team
8  @version 1.0 01/06/25
9  */
10
11 // Built-in LED pin
12 #define LED_PIN LED_BUILTIN
13
14 void setup() {
15   // Initialize serial communication and wait
16   Serial.begin(115200);
17   for (auto startNow = millis() + 2500; !Serial
18
19   // Configure LED pin as output
20   pinMode(LED_PIN, OUTPUT);
21
22   // Startup message
23   Serial.println("- Arduino Nano R4 - Blink
24 }
25
26 void loop() {
27   // Turn on the LED, wait 1 second
28   digitalWrite(LED_PIN, HIGH);
```

To upload the sketch to the board, click the **Verify** button to compile the sketch and check for errors, then click the **Upload** button to program the device with the sketch.

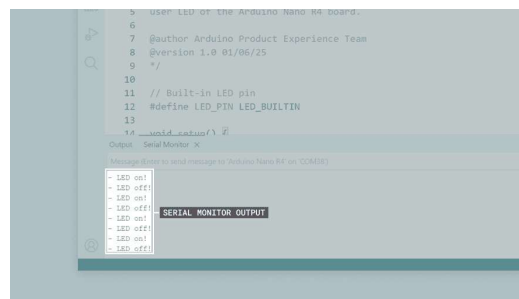


Uploading a sketch to the Nano R4 in the Arduino IDE

As shown in the animation below, you should see the built-in orange user LED of your Nano R4 board turn on for one second, then turn off for one second, repeating this cycle continuously.



Additionally, you can open the Arduino IDE's Serial Monitor (Tools > Serial Monitor) to see the status messages that the example sketch sends each time the LED state changes.



Arduino IDE Serial Monitor output for the Blink sketch

This example confirms the following:

- The Nano R4 board is correctly connected
- The Arduino IDE is properly configured
- The board is functioning correctly
- USB communication is working
- Digital pins respond to commands

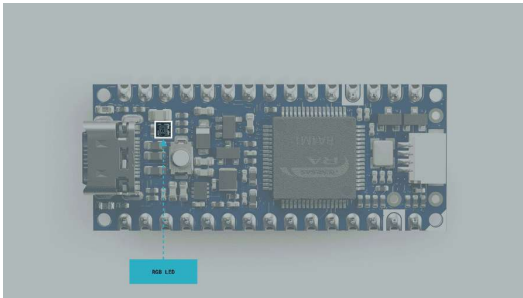
Congratulations! You have successfully completed your first program on the Nano R4 board. You are now ready to explore the more advanced features of this tiny but powerful board.

LEDs

This user manual section covers the Nano R4 built-in LEDs, showing their main hardware and software characteristics.

RGB LED

The Nano R4 features a built-in RGB LED that can be used as a visual feedback indicator for the user.



Built-in RGB LED of the Nano R4 board

The built-in RGB LED can be accessed through the following macro definitions:

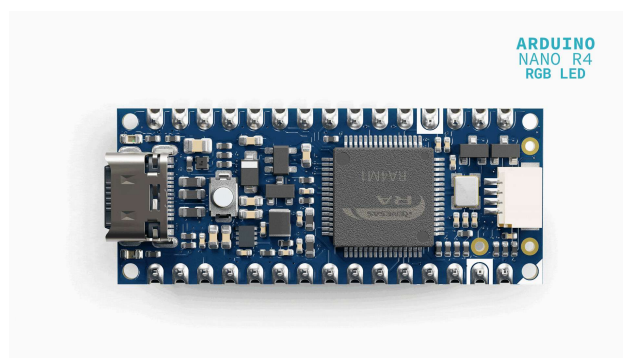
Built-in LED	Macro Definition	Microcontroller Pin
Red LED	LEDR	P409
Green LED	LEDG	P411
Blue LED	LEDB	P410

The built-in RGB LED on the Nano R4 must be pulled to ground (`GND`) to make it light up. This means that a **i** voltage level of `LOW` on each of their pins will turn the specific color of the LED on, and a voltage level of `HIGH` will turn them off.

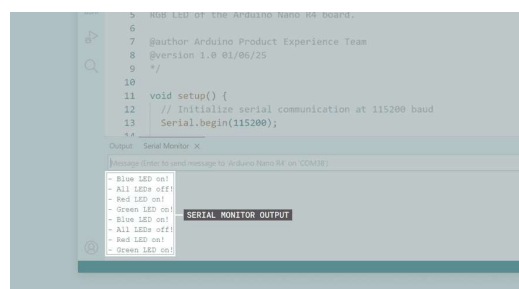
The following example sketch each of the RGB LED colors at an interval of 500 ms:

```
1  /**
2  RGB LED Example for the Arduino Nano R4 Board
3  Name: nano_r4_rgb_led.ino
4  Purpose: This sketch demonstrates how to control the
5  RGB LED of the Arduino Nano R4 board.
6
7  @author Arduino Product Experience Team
8  @version 1.0 01/06/25
9  */
10
11 void setup() {
12   // Initialize serial communication and wait until connected.
13   Serial.begin(115200);
14   for (auto startNow = millis() + 2500; !Serial; startNow += 500);
15
16   // Initialize LEDR, LEDG and LEDB as outputs.
17   pinMode(LEDR, OUTPUT);
18   pinMode(LEDG, OUTPUT);
19   pinMode(LEDB, OUTPUT);
20
21   // Turn off all LEDs initially
22   digitalWrite(LEDR, HIGH);
23   digitalWrite(LEDG, HIGH);
24   digitalWrite(LEDB, HIGH);
25
26   Serial.println("- Arduino Nano R4 - RGB LED Example");
27 }
28
```

You should now see the built-in RGB LED cycling through red, green, and blue colors, followed by a brief moment with all LEDs off, repeating this pattern continuously.



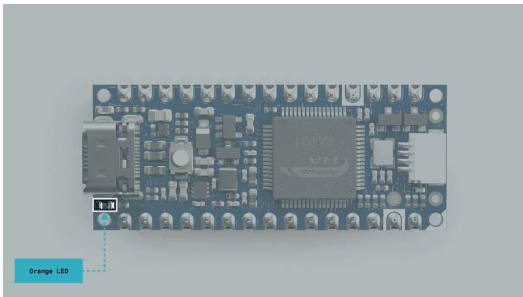
Additionally, you can open the Arduino IDE's Serial Monitor (Tools > Serial Monitor) to see the status messages that the example sketch sends each time the RGB LEDs state changes.



Arduino IDE Serial Monitor output for the RGB LED example sketch

Orange LED

The Nano R4 also features a built-in orange user LED that can be used for basic status indications and debugging purposes.



Built-in user LED of the Nano R4 board

The built-in user LED can be accessed through the following macro definition:

Built-in LED	Macro Definition	Microcontroller Pin
Orange User LED	LED_BUILTIN	P204

i

Unlike the RGB LED, the built-in user LED on the Nano R4 operates with standard logic levels. This means that a voltage level of HIGH will turn the LED on, and a voltage level of LOW will turn it off.

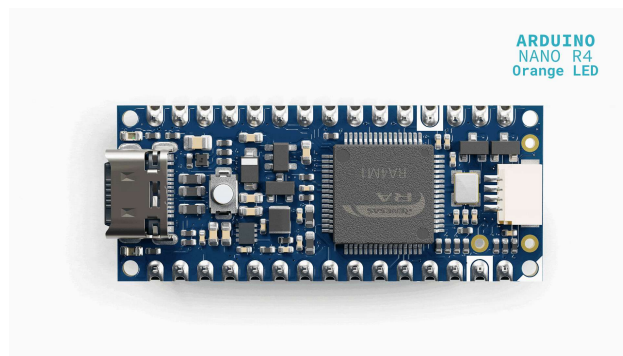
The following example sketch demonstrates how to control the built-in user LED:

```

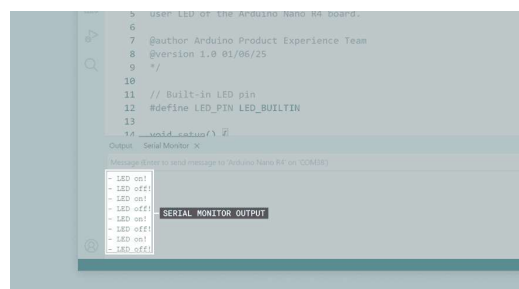
1  /**
2  User LED Example for the Arduino Nano R4 Board
3  Name: nano_r4_user_led.ino
4  Purpose: This sketch demonstrates how to control the
5  user LED of the Arduino Nano R4 board.
6
7  @author Arduino Product Experience Team
8  @version 1.0 01/06/25
9  */
10
11 void setup() {
12   // Initialize serial communication and wait until port is open
13   Serial.begin(115200);
14   for (auto startNow = millis() + 2500; !Serial.available(); startNow = millis())
15     continue;
16   // Configure LED_BUILTIN pin as output
17   pinMode(LED_BUILTIN, OUTPUT);
18
19   // Turn off LED initially
20   digitalWrite(LED_BUILTIN, LOW);
21
22   Serial.println("- Arduino Nano R4 - User LED Example");
23 }
24
25 void loop() {
26   // Turn on the built-in user LED
27   digitalWrite(LED_BUILTIN, HIGH);
28   Serial.println("- User LED on!");
29   delay(1000);
30
31   // Turn off the built-in user LED
32   digitalWrite(LED_BUILTIN, LOW);
33   Serial.println("- User LED off!");
34   delay(1000);
35 }

```

You should now see the built-in orange user LED blinking on and off at 1-second intervals, repeating this pattern continuously.



Additionally, you can open the Arduino IDE's Serial Monitor (Tools > Serial Monitor) to see the status messages that the example sketch sends each time the user LED state changes.

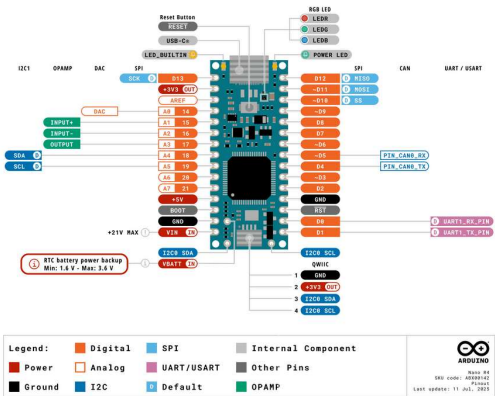


Pins

This user manual section provides comprehensive information about the Nano R4's pin capabilities and functionality. Understanding the board's pins capabilities and configurations is important for making the most of your projects with the Nano R4 board.

Pins Overview

The Nano R4 features a total of **20 accessible pins** arranged in the classic Nano form factor, maintaining compatibility with existing Nano shields and breadboard layouts. These pins provide various functionalities including digital I/O, analog input, PWM output and several communication protocols.



Nano R4 pinout overview

Pins Specifications and Characteristics

The Nano R4's pins are organized into the following categories:

Pin Type	Count	Pin Numbers	Primary Functions
Digital Pins	14	D0 - D13	Digital I/O, PWM (6 pins), SPI, UART
Analog Pins	8	A0 - A7	Analog input, Digital I/O, I ² C, DAC (A0)
Power Pins	4	VIN , 5V , 3V3 , GND	Power supply and ground
Special	2	RESET ,	System control and

The Nano R4 offers several advanced pin capabilities including multi-function pins that can serve multiple purposes depending on your project needs, native +5 VDC operation for compatibility with classic Arduino shields, internal +3.3 VDC level translation for modern sensors and electronic components via Qwiic, and built-in advanced peripherals such as DAC, CAN bus and operational amplifiers on specific pins of the board.

i

Important voltage compatibility Note: Unlike many other boards in the Nano family (Nano 33 BLE Sense, Nano 33 IoT, Nano ESP32) that operate at +3.3 VDC, the Nano R4 operates at +5 VDC. This fundamental difference affects ALL digital and analog pins, including communication interfaces like I²C (A4/A5), SPI, and UART. Before connecting any sensors, modules, or shields, always verify their voltage compatibility. **Using +3.3 VDC devices without proper level shifting can result in permanent damage to those components.**

The following table shows the electrical specifications and operating limits for all pins on the Nano R4 board:

Specification	Value	Notes
Operating Voltage	+5 VDC	Logic level for all digital pins
Input Voltage Range	0 - +5.5 VDC	+5 VDC tolerant inputs
Max Current per Pin	8 mA	Source/sink current limit
Max Total Current	200 mA	Combined current for all pins
Analog Reference	+5 VDC	Default AREF voltage

i

Important safety considerations when working with the Nano R4 pins: Never exceed +5.5 VDC on any pin to avoid permanent damage, respect the 8 mA per pin and 200 mA total current limits, handle the board with proper anti-static precautions, avoid connecting pins directly to ground or power and always verify voltage levels when connecting +3.3 VDC devices.

Digital Pins

The Nano R4 features 14 digital pins (D0 to D13) that can be configured as either digital inputs or digital outputs. These pins operate at +5 VDC logic levels and can source or sink up to 8 mA of current per pin. Digital pins are the foundation of most Arduino projects, allowing you to control LEDs, read button states, interface with sensors and communicate with other devices.

Arduino Pin	Microcontroller Pin	Additional Functions	Special Features
D0	P104	UART RX, PWM	Serial communication
D1	P105	UART TX, PWM	Serial communication
D2	P213	PWM	External interrupt
D3	P212	PWM	External interrupt
D4	P109	CAN TX, PWM	CAN communication
D5	P110	CAN RX, PWM	CAN communication
D6	P107	PWM	Digital I/O
D7	P106	PWM	Digital I/O
D8	P300	PWM	Digital I/O
D9	P108	PWM	Digital I/O
D10	P103	SPI CS, PWM	SPI communication
D11	P101	SPI MOSI, PWM	SPI communication
D12	P100	SPI MISO	SPI communication
D13	P102	SPI SCK	SPI communication

Important note: Pins `D0` and `D1` are used for serial communication (UART) and should be avoided for general digital I/O when using Serial communication. Pins `D4` and `D5` can be used for CAN bus communication. Pins `D10`, `D11`, `D12` and `D13` are used for SPI communication.

The Nano R4's digital pins offer the following specifications:

Specification	Value	Notes
Logic Voltage	+5 VDC	<code>HIGH</code> and <code>LOW</code> logic levels
Input Voltage	0 to +5.5 VDC	+5 VDC tolerant inputs
Max Current (Source)	8 mA	Per pin source current
Max Current (Sink)	8 mA	Per pin sink current
Total Max Current	200 mA	Combined for all pins
Input Resistance	20-50 kΩ	Internal pull-up resistor

Specification	Value	Notes
Digital <code>LOW</code>	0 to +1.5 VDC	Maximum voltage for <code>LOW</code>

Digital pins can be configured and controlled using the following basic Arduino functions.

You can configure a pin's mode using the `pinMode()` function:

```
1 pinMode(pin, mode);
```

To write a digital value to an output pin, use the `digitalWrite()` function:

```
1 digitalWrite(pin, value);
```

To read the state of a digital input pin, use the `digitalRead()` function:

```
1 digitalRead(pin);
```

The available pin modes are `OUTPUT` for digital output, `INPUT` for digital input with high impedance, and `INPUT_PULLUP` for digital input with internal pull-up resistor enabled. Digital output values can be `HIGH` (+5 VDC) or `LOW` (0 VDC), and digital input readings will return `HIGH` or `LOW` based on the voltage level detected on the pin.

 The following example demonstrate basic digital pin functionality using simple connections that you can easily test with the Nano R4 board.

The following example demonstrates using both digital input and output simultaneously by reading a button and controlling the built-in LED of the board:

```

1  /**
2  Combined Digital I/O Example for the Arduino
3  Name: nano_r4_digital_io_combined.ino
4  Purpose: This sketch demonstrates reading a
5  the built-in LED state each time the button
6
7  @author Arduino Product Experience Team
8  @version 1.0 01/06/25
9  */
10
11 // Pin definitions
12 const int buttonPin = 2;           // Button pin
13 const int ledPin = LED_BUILTIN;    // Built-in LED pin
14
15 // Variables to store button and LED state
16 int buttonState = 0;
17 int lastButtonState = HIGH;
18 bool ledState = false;
19
20 void setup() {
21   // Initialize serial communication and wait for connection
22   Serial.begin(115200);
23   for (auto startNow = millis() + 2500; !Serial.available(); startNow = millis())
24     continue;
25   // Configure pins
26   pinMode(buttonPin, INPUT_PULLUP);
27   pinMode(ledPin, OUTPUT);
28
29   // Print a message to the serial monitor
30   Serial.println("Digital I/O Example");
31 }
32
33 void loop() {
34   // Read the state of the push button
35   int buttonState = digitalRead(buttonPin);
36
37   // Check if the button state has changed
38   if (buttonState != lastButtonState) {
39     // Toggle the LED state
40     ledState = !ledState;
41     digitalWrite(ledPin, ledState);
42     lastButtonState = buttonState;
43   }
44 }

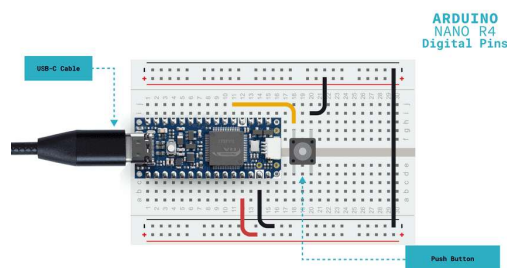
```

To test this example, connect a push button to the Nano R4 board as follows:

Connect one leg of a push button to pin **D2**

Connect the other leg of the push button to **GND**

No external components needed (using built-in LED and internal pull-up)



Digital pins test circuit on the Nano R4 board

You should now see the built-in LED toggle on and off each time you press the button. The LED will stay in its current state until you press the button again. Additionally, you can open the Arduino IDE's Serial Monitor (Tools > Serial Monitor) to see messages indicating when the button is pressed and the current LED state.



Arduino IDE Serial Monitor output for the combined digital I/O example sketch

Analog Pins

The Nano R4 features **8 analog input pins** (A0 to A7) that can be read using the `analogRead()` function. These pins allow you to measure continuously varying voltages, making them perfect for reading sensors like potentiometers, light sensors, temperature sensors and other analog components and devices. The analog-to-digital converter (ADC) built into the RA4M1 microcontroller of the Nano R4 board converts the analog voltage into a digital value that your sketch can process.

The Nano R4 analog pins provide the following functionality:

Arduino Pin	Microcontroller Pin	Additional Functions	Special Features
A0	P000	DAC	12-bit DAC output
A1	P001	OPAMP +	Operational amplifier
A2	P002	OPAMP -	Operational amplifier
A3	P003	OPAMP OUT	Operational amplifier
A4	P004	SDA (I ² C)	I ² C communication
A5	P010	SCL (I ² C)	I ² C communication
A6	P014	Analog In	Analog input only
A7	P015	Analog In	Analog input only

Important note: Pin A0 has a built-in 12-bit Digital-to-Analog Converter (DAC) for analog output. Pins A1, A2 and A3 are connected to the integrated operational

The Nano R4's analog pins offer the following specifications:

Specification	Value	Notes
Input Voltage	0 to +5 VDC	Maximum safe input voltage
Default Resolution	10-bit	Values 0-1023
Maximum Resolution	14-bit	Values 0-16383
Default Reference	+5 VDC	AREF voltage
Internal Reference	+1.5 VDC	Built-in precision reference
Sample Rate	Up to 1 MSPS	Maximum sampling speed
Accuracy	±2 LSB	Typical conversion accuracy

You can read analog values using the `analogRead()` function:

```
1 value = analogRead(pin);
```

The default reference voltage of these pins is +5 VDC, but this can be changed using the `analogReference()` function. You can use `analogReference(AR_DEFAULT)` for the default reference of +5 VDC or `analogReference(AR_INTERNAL)` for the built-in reference of +1.5 VDC.

The default resolution is set to 10-bit, but it can be updated to 12-bit and 14-bit resolutions using the `analogReadResolution()` function in the `setup()` of your sketch. Available options are `analogReadResolution(10)` for default 10-bit, `analogReadResolution(12)` for 12-bit, or `analogReadResolution(14)` for maximum 14-bit resolution.

 The following examples demonstrate basic analog pin functionality using simple connections that you can easily test with the Nano R4 board.

The following example demonstrates how to read an analog value and display it on the Serial Monitor:

```

1  /**
2  Analog Input Example for the Arduino Nano R4
3  Name: nano_r4_analog_input.ino
4  Purpose: This sketch demonstrates how to read an analog value
5  and display the value on the Serial Monitor
6
7  @author Arduino Product Experience Team
8  @version 1.0 01/06/25
9  */
10
11 // Analog input pin
12 const int analogPin = A0;
13
14 void setup() {
15   // Initialize serial communication and wait until port is open
16   Serial.begin(115200);
17   for (auto startNow = millis() + 2500; !Serial; startNow++) {}
18
19   Serial.println("- Arduino Nano R4 - Analog Input Example");
20   Serial.println("- Reading analog values from pin A0");
21 }
22
23 void loop() {
24   // Read the analog value (0 - 1023 with 5V)
25   int analogValue = analogRead(analogPin);
26
27   // Convert to voltage (0 to +5 VDC)
28   float voltage = analogValue * (5.0 / 1023);

```

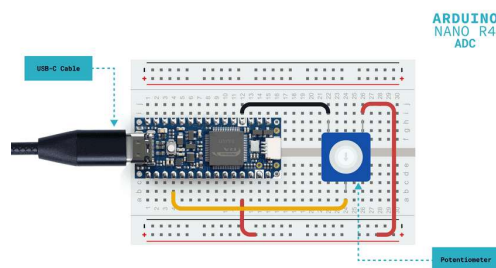
To test this example, connect a potentiometer to the Nano R4 board as follows:

Connect the middle pin of a potentiometer to A0

Connect one outer pin of the potentiometer to +5 VDC

Connect the other outer pin of the potentiometer to

GND



ADC test circuit on the Nano R4 board

You can open the Arduino IDE's Serial Monitor (Tools > Serial Monitor) to see the real-time analog values and voltage measurements as you adjust the potentiometer. As you turn the potentiometer, the values will range from 0 to 1023, with corresponding voltage readings from 0 to +5 VDC.

```

30 Serial.print("- Analog Value: ");
31 Serial.print(analogValue);
32 Serial.print(" | Voltage: ");
33 Serial.print(voltage, 2);
34 Serial.println(" VDC");
35
36 // Wait half a second before next reading
37 delay(500);
38 }

```

Output: Serial Monitor x

Message (Click to send message to Arduino Nano RP on COM8)

Analog Reader	Raw	Voltage	%	VDC
- Analog Value	723	Voltage:	5.83	VDC
- Analog Value	423	Voltage:	3.04	VDC
- Analog Value	523	Voltage:	2.56	VDC
- Analog Value	423	Voltage:	2.07	VDC
- Analog Value	323	Voltage:	1.58	VDC
- Analog Value	223	Voltage:	1.09	VDC
- Analog Value	123	Voltage:	0.60	VDC

SERIAL MONITOR OUTPUT

Arduino IDE Serial Monitor output for the analog input example sketch

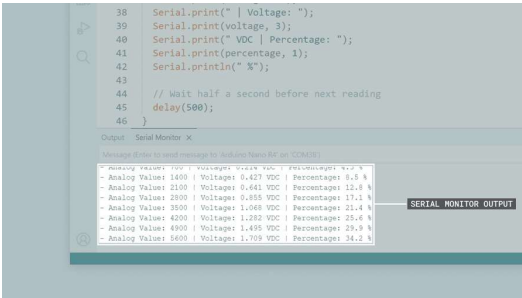
The following example demonstrates how to use 14-bit resolution for more precise analog readings; **use the same potentiometer connection from the first example:**

```

1  /**
2  High-Resolution Analog Input Example for the
3  Name: nano_r4_analog_high_res.ino
4  Purpose: This sketch demonstrates how to use
5  for precise analog input measurements.
6
7  @author Arduino Product Experience Team
8  @version 1.0 01/06/25
9  */
10
11 // Analog input pin
12 const int analogPin = A0;
13
14 void setup() {
15   // Initialize serial communication and wait until connected
16   Serial.begin(115200);
17   for (auto startNow = millis() + 2500; !Serial.available(); startNow = millis());
18
19   // Set analog read resolution to 14-bit (default is 10-bit)
20   analogReadResolution(14);
21
22   Serial.println("- Arduino Nano R4 - High-Resolution Analog Input Example");
23   Serial.println("- Using 14-bit resolution");
24 }
25
26 void loop() {
27   // Read the analog value (0 - 16383 with 14-bit resolution)
28   int analogValue = analogRead(analogPin);
29
30   // Calculate voltage and percentage
31   float voltage = (analogValue * 5.0) / 16383;
32   float percentage = (analogValue * 100) / 16383;
33
34   // Print the results
35   Serial.print("- Analog Value: ");
36   Serial.print(analogValue);
37   Serial.print(" | Voltage: ");
38   Serial.print(voltage, 2);
39   Serial.print(" | Percentage: ");
40   Serial.print(percentage, 1);
41   Serial.println("%");
42
43   // Wait 1 second before next reading
44   delay(1000);
45 }

```

You can open the Arduino IDE's Serial Monitor (Tools > Serial Monitor) to see the high-resolution analog values, voltage measurements and percentage calculations as you adjust the potentiometer. With 14-bit resolution, the values will range from 0 to 16383 instead of the standard 0 to 1023, providing significantly higher precision for sensitive measurements.



Arduino IDE Serial Monitor output for the high-resolution analog input example sketch

PWM (Pulse Width Modulation)

The Nano R4 board features multiple pins with PWM capability that can be used to generate analog-like output signals. PWM works by rapidly switching a digital output between **HIGH** and **LOW** states, where the ratio of **HIGH** time to the total period determines the effective analog voltage output.

PWM Compatibility Note: The Nano R4 shares identical PWM capabilities with the UNO R4 Minima, as both boards use the same RA4M1 microcontroller. This includes the **i** same default 8-bit resolution, maximum 16-bit resolution, and PWM frequency control features. Code written for PWM on the UNO R4 Minima will work identically on the Nano R4.

The Nano R4 board provides PWM functionality on the following pins:

Arduino Pin	Microcontroller Pin	PWM Channel	Primary Function
D3	P212	Channel 0B	Digital I/O
D5	P110	Channel 1B	Digital I/O
D6	P107	Channel 0A	Digital I/O
D9	P108	Channel 0B	Digital I/O
D10	P103	Channel 2A	Digital I/O, SPI CS
D11	P101	Channel 5A	Digital I/O, SPI MOSI

Important notes: Pins **D3** and **D9** share the same PWM channel (**Channel 0B**) and cannot be used for PWM simultaneously. When using PWM on one of these pins, the other cannot output an independent PWM signal. **i** Similarly, pins **D6**, **D3**, and **D9** all use timer **GPT0**, which means they will share the same PWM frequency

The Nano R4's PWM offers the following technical specifications:

Specification	Value	Notes
Default Resolution	8-bit (0-255)	Standard Arduino compatibility
Maximum Resolution	16-bit (0-65535)	Full precision control

You can use PWM pins as analog output pins with the `analogWrite()` function:

```
1 analogWrite(pin, value);
```

By default, the resolution is 8-bit (0 to 255). You can use the `analogWriteResolution()` resolution to change this, supporting up to 16-bit (0 to 65535) resolution:

```
1 analogWriteResolution(resolution);
```

The Nano R4 supports the following PWM resolution options that you can configure based on your project's precision requirements:

`analogWriteResolution(8)` : 8-bit resolution (0 to 255, **default resolution**)

`analogWriteResolution(10)` : 10-bit resolution (0 to 1023)

`analogWriteResolution(12)` : 12-bit resolution (0 to 4095)

`analogWriteResolution(14)` : 14-bit resolution (0 to 16383)

`analogWriteResolution(16)` : 16-bit resolution (0 to 65535, **maximum resolution**)

 The following PWM examples use the built-in orange user LED (`LED_BUILTIN`) of the Nano R4 board, which supports PWM for brightness control. This eliminates the need for external components and allows you to test PWM functionality immediately.

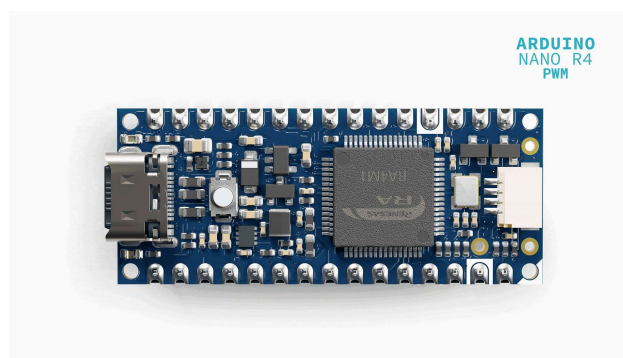
The following example demonstrates how to control the brightness of the built-in orange user LED using PWM:

```

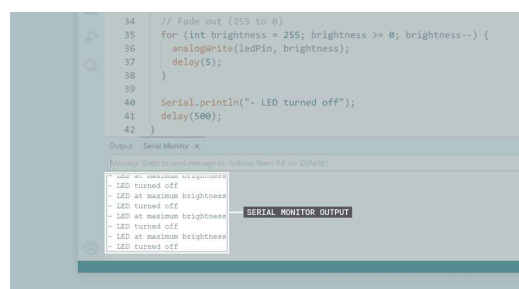
1  /**
2  PWM Example for the Arduino Nano R4 Board
3  Name: nano_r4_pwm_led.ino
4  Purpose: This sketch demonstrates how to use
5  the brightness of the built-in user LED of
6
7  @author Arduino Product Experience Team
8  @version 1.0 01/06/25
9  */
10
11 // Built-in LED pin (supports PWM)
12 const int ledPin = LED_BUILTIN;
13
14 void setup() {
15   // Initialize serial communication and wait
16   Serial.begin(115200);
17   for (auto startNow = millis() + 2500; !Serial.available(); startNow++) {}
18
19   // No need to set pinMode for PWM pins -
20
21   Serial.println("- Arduino Nano R4 - PWM LED Example");
22   Serial.println("- Built-in LED will fade in and out");
23 }
24
25 void loop() {
26   // Fade in (0 to 255)
27   for (int brightness = 0; brightness <= 255; brightness++) {
28     analogWrite(ledPin, brightness);
29     delay(5);
30   }
31   // Fade out (255 to 0)
32   for (int brightness = 255; brightness >= 0; brightness--) {
33     analogWrite(ledPin, brightness);
34     delay(5);
35   }
36   Serial.println("- LED turned off");
37   delay(500);
38 }

```

You should now see the built-in orange user LED of your Nano R4 board gradually fade in and out, creating a smooth breathing effect that repeats continuously.



Additionally, you can open the Arduino IDE's Serial Monitor (Tools > Serial Monitor) to see the status messages that the example sketch sends at key brightness levels.



Arduino IDE Serial Monitor output for the PWM example sketch

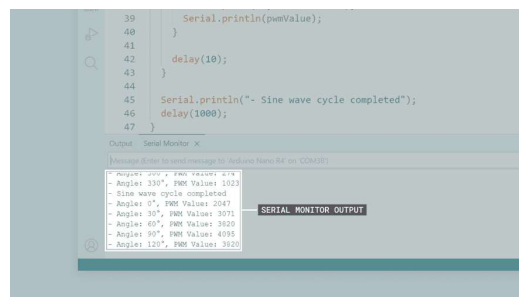
The following example demonstrates how to use a 12-bit PWM resolution for more precise control of the built-in orange user LED:

```

1  /**
2  High-Resolution PWM Example for the Arduino
3  Name: nano_r4_pwm_high_res.ino
4  Purpose: This sketch demonstrates how to use a 12-bit
5  for precise control of the built-in orange user LED
6
7  @author Arduino Product Experience Team
8  @version 1.0 01/06/25
9  */
10
11 // Built-in LED pin (supports PWM)
12 const int pwmPin = LED_BUILTIN;
13
14 void setup() {
15   // Initialize serial communication and wait for the serial
16   Serial.begin(115200);
17   for (auto startNow = millis() + 2500; !Serial.available(); startNow = millis()) {}
18
19   // Set PWM resolution to 12-bit (0-4095)
20   analogWriteResolution(12);
21
22   Serial.println("- Arduino Nano R4 - High-Resolution PWM Example");
23   Serial.println("- Using 12-bit resolution");
24 }
25
26 void loop() {
27   // Generate a smooth sine wave using 12-bit PWM resolution
28   for (int i = 0; i < 360; i++) {
29     // Calculate the sine wave value (0 to 4095)
30     int pwmValue = (int) (sin((i * 3.14159) / 180) * 4095 + 2048);
31     // Write the PWM value to the built-in LED pin
32     analogWrite(pwmPin, pwmValue);
33     // Print the angle and PWM value to the serial monitor
34     Serial.print(i);
35     Serial.print(" - Angle: ");
36     Serial.print(pwmValue);
37     Serial.println(" PWM Value");
38     // Delay for 10ms between each PWM value
39     delay(10);
40   }
41   // Print a message to the serial monitor when the sine wave cycle is completed
42   Serial.println("- Sine wave cycle completed");
43   // Delay for 1000ms before starting the next cycle
44   delay(1000);
45 }

```

This high-resolution example creates a smooth sine wave pattern with the built-in LED brightness, demonstrating the precision available with a 12-bit PWM resolution. You should see a very smooth transition in the LED brightness following a sine wave pattern. Additionally, you can open the Arduino IDE's Serial Monitor (Tools > Serial Monitor) to see the angle and PWM value outputs that demonstrate the precise 12-bit control values being used.



Operational Amplifier (OPAMP)

The Nano R4 board features a built-in operational amplifier (OPAMP) that provides signal conditioning and amplification capabilities directly on the board. The OPAMP is connected to analog pins `A1`, `A2` and `A3`, allowing you to perform analog signal processing without requiring external amplifier circuits. This feature is particularly useful for sensor signal amplification, buffering and analog filtering applications.

The Nano R4 board OPAMP provides the following pin connections:

Arduino Pin	Microcontroller Pin	OPAMP Function	Description
<code>A1</code>	<code>P001</code>	OPAMP +	Non-inverting input (+)
<code>A2</code>	<code>P002</code>	OPAMP -	Inverting input (-)
<code>A3</code>	<code>P003</code>	OPAMP OUT	Amplifier output

Important note: When using the OPAMP functionality, pins `A1`, `A2` and `A3` cannot simultaneously be used as regular analog inputs. The positive supply (Vs+) is fixed to approximately +5 VDC and the negative supply (Vs-) is fixed to `GND`.

The Nano R4's OPAMP offers the following electrical characteristics:

Parameter	Low-Speed Mode	High-Speed Mode	Unit	Notes
Common Input Voltage Range	+1.8 to +5.5	+0.3 to +4.4	VDC	Typical with 5V supply
Output Voltage Range	+0.1 to +4.9	+0.1 to +4.9	VDC	Typical with 5V supply
Input Offset Voltage	-10 to +10	-10 to +10	mV	-
Open Loop Gain	60 (typical)	125 (typical)	dB	-
Gain-Bandwidth Product	-	1.7	MHz	High-speed mode
Slew Rate	0.12	-	V/μs	CL = 20 pF
Settling Time	650	13	μs	To 1% accuracy
Load Current	-100 to +100	-100 to +100	μA	Maximum

Parameter	Low-Speed Mode	High-Speed Mode	Unit	Notes
Load Capacitance	20	20	pF	Maximum

You can configure and use the OPAMP using the dedicated `OPAMP.h` library, which is included in the Arduino UNO R4 Boards core. To startup the OPAMP, simply include the library and call `OPAMP.begin(speed)` where speed can be `OPAMP_SPEED_LOWSPEED` for lower power consumption or `OPAMP_SPEED_HIGHSPEED` for better performance.

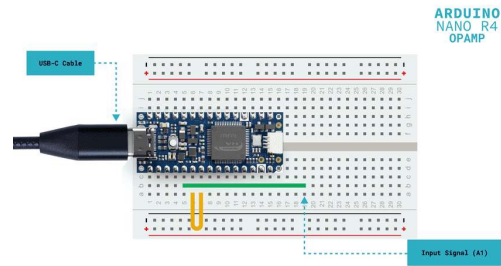
The following example demonstrates how to initialize and use the OPAMP. The same code works for both voltage follower and amplifier configurations, **the difference is only in the external connections:**

```
1 /**
2  OPAMP Example for the Arduino Nano R4 Board
3  Name: nano_r4_opamp_example.ino
4  Purpose: This sketch demonstrates how to in
5  Works for both voltage follower and amplifi
6
7  @author Arduino Product Experience Team
8  @version 1.0 01/06/25
9  */
10
11 #include <OPAMP.h>
12
13 void setup() {
14   // Initialize serial communication and wa
15   Serial.begin(115200);
16   for (auto startNow = millis() + 2500; !Se
17
18   Serial.println("- Arduino Nano R4 - OPAMP
19   Serial.println("- Initializing OPAMP in h
20
21   // Initialize OPAMP in high-speed mode
22   OPAMP.begin(OPAMP_SPEED_HIGHSPEED);
23
24   Serial.println("- OPAMP initialized succe
25   Serial.println("- OPAMP is now ready for
26   Serial.println("- The behavior depends or
27 }
28
```

To configure the OPAMP as a **voltage follower**, connect the Nano R4 board as follows:

- Connect pin `A2` (-) to pin `A3` (Output) with a jumper wire
- Connect the input signal to pin `A1` (+)

For testing, connect pin `A1` to `GND` to see 0 VDC output, or



Voltage follower test circuit on the Nano R4 board

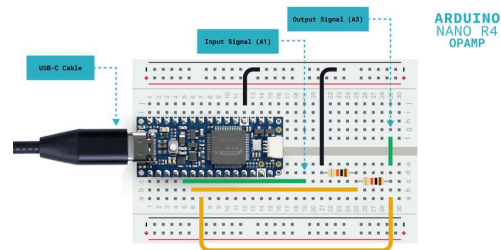
In voltage follower configuration, any voltage applied at **A1** should be mirrored onto **A3**. This provides a high-impedance buffer that doesn't load down the input signal source.

The Nano R4 OPAMP can also be configured as a **non-inverting amplifier** to amplify small signals. For example, a simple 2x amplifier can be built using two 10k Ω resistors as follows:

Connect one 10k Ω resistor between **A2** (-) and **GND**

Connect another 10k Ω resistor between **A3** (Output) and **A2** (-)

For testing, apply an input signal to **A1** (+).



Non inverting amplifier test circuit on the Nano R4 board

The output at **A3** will be double the amplitude of the input signal.

In a non-inverting amplifier configuration, remember that the input signal and the Nano R4 should share the same GND, and the amplified output signal should not exceed approximately +4.7 VDC to avoid clipping.

Digital-to-Analog Converter (DAC)

The Nano R4 features a built-in 12-bit Digital-to-Analog Converter (DAC) connected to pin **A0**. Unlike PWM pins that

applications requiring precise analog signals, such as audio generation, sensor calibration, control systems and waveform generation.

The Nano R4 DAC provides the following functionality:

Specification	Value	Notes
Resolution	12-bit	4096 discrete output levels
Output Pin	A0	Dedicated DAC output pin
Output Voltage Range	+0.35 to +4.5 VDC	Typical range with +5 VDC supply
Default Resolution	8-bit	0 to 255
Maximum Resolution	12-bit	0 to 4095
Output Impedance	5Ω (typical)	Low impedance output
Conversion Time	Max 30 μs	Time to update output voltage
Resistive Load	Min 30 kΩ	Minimum recommended load
Load Capacitance	Max 50 pF	Maximum capacitive load

Important note: When using the DAC on pin A0, this pin cannot simultaneously be used as an analog input. The DAC provides true analog output, making it superior to PWM for applications requiring smooth, continuous voltage levels.

The Nano R4's DAC offers the following technical specifications:

Parameter	Min	Typ	Max	Unit	Notes
Differential Nonlinearity (DNL)	-	±0.5	±2.0	LSB	DNL error
Integral Nonlinearity (INL)	-	±2.0	±16.0	LSB	INL error
Offset Error	-	-	±30	mV	Output offset
Full-Scale Error	-	-	±30	mV	Full-scale accuracy

You can write analog values to the DAC using the `analogWrite()` function:

```
1 analogWrite(DAC, value);
```

The default resolution is 8-bit (0 to 255), but this can be

`analogWriteResolution(10)` for 10-bit resolution or
`analogWriteResolution(12)` for maximum 12-bit
 resolution.

The DAC reference voltage depends on the selected reference mode, and the output voltage is calculated as:

$$\text{Output Voltage} = (\text{DAC_Value} / 4095) \times \text{Reference_Voltage}$$

. The output voltage range is typically from +0.35 VDC to +4.5 VDC when using the supply voltage as reference.

 The following examples demonstrate basic DAC functionality that you can easily test with the Nano R4 board.

The following example demonstrates how to generate a simple voltage output using the DAC. **No external components are needed for this example:**

```

1  /**
2  DAC Basic Output Example for the Arduino Nano R4
3  Name: nano_r4_dac_basic.ino
4  Purpose: This sketch demonstrates how to use the DAC to generate
5  precise analog voltages on pin A0.
6
7  @author Arduino Product Experience Team
8  @version 1.0 01/06/25
9  */
10
11 void setup() {
12   // Initialize serial communication and wait until ready
13   Serial.begin(115200);
14   for (auto startNow = millis() + 2500; !Serial; startNow++) {}
15
16   // Set DAC resolution to 12-bit for maximum precision
17   analogWriteResolution(12);
18
19   Serial.println("- Arduino Nano R4 - DAC Example");
20   Serial.println("- Generating precise voltages");
21   Serial.println("- Connect a multimeter to pin A0");
22 }
23
24 void loop() {
25   // Generate different voltage levels
26   // 0, +1.25, +2.5, +3.75 and +5 VDC
27   int dacValues[] = {0, 1024, 2048, 3072, 4095};
28   float voltages[] = {0.0, 1.25, 2.5, 3.75, 5.0};
29   for (int i = 0; i < 5; i++) {
30     analogWrite(dacValues[i]);
31     delay(2000);
32   }
33 }

```

To test this example, connect a digital multimeter between pin `A0` and `GND` to measure the output voltage. You should see the voltage change in precise steps every two seconds, showing the DAC's ability to generate exact analog voltages. For best results, connect an oscilloscope to pin `A0` to visualize the step output.



Step output oscilloscope output

You can open the Arduino IDE's Serial Monitor (Tools > Serial Monitor) to see the DAC values and corresponding target voltages. The output should closely match the calculated voltages with high precision.

The following example demonstrates how to generate a smooth sine wave using the DAC. **No external components are needed for this example:**

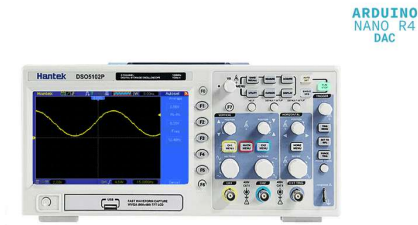
```

1  /**
2  DAC Sine Wave Generator for the Arduino Nano R4
3  Name: nano_r4_dac_sine_wave.ino
4  Purpose: This sketch generates a smooth sine wave for testing and signal generation applications.
5  for testing and signal generation applications.
6
7  @author Arduino Product Experience Team
8  @version 1.0 01/06/25
9  */
10
11 void setup() {
12   // Initialize serial communication and wait for the serial monitor to open
13   Serial.begin(115200);
14   for (auto startNow = millis() + 2500; !Serial.available(); startNow = millis()) {}
15
16   // Set DAC resolution to 12-bit for smooth output
17   analogWriteResolution(12);
18
19   Serial.println("- Arduino Nano R4 - DAC Sine Wave Generator");
20   Serial.println("- Generating sine wave on pin A0");
21   Serial.println("- Connect an oscilloscope to pin A0 to visualize the output");
22 }
23
24 void loop() {
25   // Generate one complete sine wave cycle
26   for (int angle = 0; angle < 360; angle++) {
27     // Calculate sine value (-1 to +1) and convert to DAC value
28     float sineValue = sin(angle * PI / 180);
29     int dacValue = map(sineValue, -1, 1, 0, 4095);
30     analogWrite(dacValue);
31     delay(10);
32   }
33 }

```

You can open the Arduino IDE's Serial Monitor (Tools > Serial Monitor) to see the sine wave generation progress with angle, DAC values, and corresponding voltages.

For best results, connect an oscilloscope to pin `A0` to visualize the smooth sine wave output.



Sine wave output oscilloscope output

Real-Time Clock (RTC)

The Nano R4 features a built-in Real-Time Clock (RTC) that allows your projects to keep track of date and time. The RTC is integrated within the RA4M1 microcontroller and can maintain accurate timekeeping for applications that require scheduling, data logging or time-based events. With optional backup power support via the board's `VBATT` pin, the RTC can continue running even when the main power is disconnected, using a 100-year calendar from 2000 to 2099 that automatically adjusts dates for leap years.

The RTC is particularly useful when your project needs to know the actual date and time, rather than just measuring time intervals. While Arduino functions like `millis()` and `delay()` are perfect for timing operations (like blinking an LED every second), the RTC is essential for applications like data loggers that need timestamps, alarm clocks, scheduling systems or any project that needs to know "what time is it right now?" even after being powered off and on again.

The Nano R4's RTC offers the following technical specifications:

Parameter	Value	Notes
Time Range	2000 to 2099	100-year calendar
Backup Voltage	1.6V to 3.6V	Via VBATT pin
Current Consumption	~1 μ A	In backup mode
Temperature Range	-40°C to +85°C	Operating range

You can configure and use the RTC using the dedicated `RTC.h` library, which is included in the Arduino UNO R4 Boards core. The library provides functions to set time, get time, check RTC status and configure alarms.

The following example demonstrates how to initialize the RTC and display the current date and time:

```

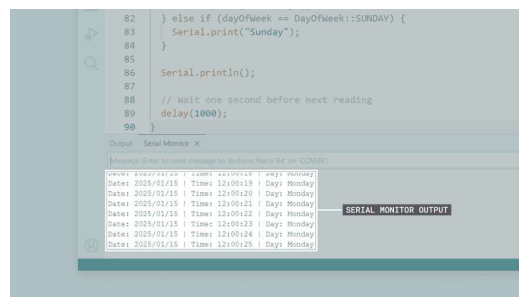
1  /**
2  RTC Basic Example for the Arduino Nano R4 F
3  Name: nano_r4_rtc_basic.ino
4  Purpose: This sketch demonstrates how to use the RTC module
5  to keep track of date and time.
6
7  @author Arduino Product Experience Team
8  @version 1.0 01/06/25
9  */
10
11 #include "RTC.h"
12
13 void setup() {
14   // Initialize serial communication and wait until connected
15   Serial.begin(115200);
16   for (auto startNow = millis() + 2500; !Serial; startNow = millis());
17
18   Serial.println("- Arduino Nano R4 - RTC Basic Example");
19   Serial.println("- Initializing RTC...");
20
21   // Initialize RTC
22   RTC.begin();
23
24   // Set initial time: January 15, 2025, 12:00:00
25   RTCTime startTime(15, Month::JANUARY, 2025, 12, 0, 0);
26   RTC.setTime(startTime);
27
28   Serial.println("- RTC time has been set!");

```



To test this example, no external connections are needed.
The RTC operates internally within the microcontroller.

You can open the Arduino IDE's Serial Monitor (Tools > Serial Monitor) to see the real-time date and time updates. The display will show the current date, time, and day of the week, updating every second.



Arduino IDE Serial Monitor output for the RTC example sketch

To maintain RTC timekeeping when the main power is disconnected, you can connect a backup battery to the **VBATT** pin:

Connect the positive terminal of a 3V coin cell battery (CR2032) to the **VBATT** pin of the board

With this configuration, the board's RTC will continue running on backup power when main power is removed. You can also set the board's RTC time programmatically using the following format. Remember to update the date and time values to match the current date when you upload your sketch:

```
1 // Create RTCTime object with current date and time
2 // Format: day, month, year, hour, minute, second
3 RTCTime newTime(30, Month::JUNE, 2025, 15, 30, 0);
4
5 // Set the RTC time
6 RTC.setTime(newTime);
```

When working with the RTC on the Nano R4, there are several key points to keep in mind for successful implementation:

The RTC requires an initial time setting before it can provide accurate timekeeping, so make sure to configure the date and time when you first start your project. For applications that need continuous timekeeping even when the main power is disconnected, connecting a backup battery to the VBATT pin is highly recommended.

Keep in mind that **the RTC stores time exactly as you set it**, so you'll need to handle time zone conversions and daylight saving time adjustments in your application code if needed.

EEPROM (Non-Volatile Memory)

The Nano R4 board features built-in EEPROM (Electrically Erasable Programmable Read-Only Memory) that allows your projects to store data permanently, even when the board is powered off. The EEPROM is implemented using flash memory emulation within the RA4M1 microcontroller and provides 8 KB of non-volatile storage space for applications that need to remember settings, sensor calibrations, or user preferences between power cycles.

EEPROM is particularly useful when your project needs to remember information permanently, rather than just during program execution. While variables stored in SRAM are lost when power is removed, EEPROM retains data indefinitely. This makes it essential for applications like saving user preferences, storing sensor calibration values, keeping configuration settings, or any project that needs to "remember" data even after being unplugged and reconnected.

The Nano R4's EEPROM offers the following technical

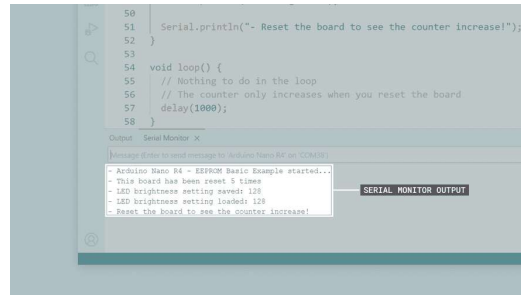
Parameter	Value	Notes
Capacity	8 KB	Data area
Memory Type	Data flash memory	SuperFlash® technology
Write Cycles	100,000	Program/erase cycles max
Programming Unit	64-bit	Minimum write unit
Erase Unit	1 KB	Minimum erase unit
Read Speed	6 clock cycles	Time to read data
Retention Time	10+ years	Data retention period
Operating Voltage	+5 VDC	Same as board

You can read and write to EEPROM using the dedicated `EEPROM.h` library, which is included in the Arduino UNO R4 Boards core. The library provides simple functions to store and retrieve individual bytes or larger data structures.

The following example demonstrates how to store and retrieve data from EEPROM:

```
1  /**
2  EEPROM Basic Example for the Arduino Nano R4
3  Name: nano_r4_eeprom_basic.ino
4  Purpose: This sketch demonstrates how to store and retrieve data
5  from the built-in EEPROM memory.
6
7  @author Arduino Product Experience Team
8  @version 1.0 01/06/25
9  */
10
11 #include <EEPROM.h>
12
13 void setup() {
14     // Initialize serial communication and wait for the serial port
15     Serial.begin(115200);
16     for (auto startNow = millis() + 2500; !Serial.available(); startNow = millis())
17         continue;
18     Serial.println("- Arduino Nano R4 - EEPROM Basic Example");
19
20     // Read a counter from EEPROM address 0
21     int bootCounter = EEPROM.read(0);
22
23     // If this is the first time (EEPROM start address 0 is 255)
24     if (bootCounter == 255) {
25         bootCounter = 0;
26         Serial.println("- First boot detected!");
27     }
28 }
```

You can open the Arduino IDE's Serial Monitor (Tools > Serial Monitor) to see the EEPROM operations. Each time you reset the board, the boot counter will increment, demonstrating that the value persists in EEPROM memory.



Arduino IDE Serial Monitor output for the EEPROM example sketch

You can also store complex data structures using the

`EEPROM.put()` and `EEPROM.get()` functions:

```
1 // Define a structure for complex data
2 struct Settings {
3   int threshold;
4   float calibration;
5   bool enabled;
6   char deviceName[16];
7 };
8
9 // Store structure to EEPROM
10 Settings mySettings = {512, 3.14, true, "Nano R4"};
11 EEPROM.put(0, mySettings);
12
13 // Read structure from EEPROM
14 Settings loadedSettings;
15 EEPROM.get(0, loadedSettings);
```

When working with EEPROM on the Nano R4, there are several key points to keep in mind for successful implementation:

The EEPROM requires careful management of write operations since flash memory has a limited number of write cycles (100,000). Avoid writing to the same address repeatedly in tight loops, as this will quickly wear out the memory. Instead, only write when values actually change, and consider spreading data across different addresses to distribute wear evenly.

Keep in mind that EEPROM stores data as individual bytes (0 to 255), so larger data types need to be broken down or use the `put()` and `get()` functions for automatic handling.

The EEPROM retains data for over 10 years under normal conditions, making it excellent for long-term storage of configuration data and user preferences.

UART Communication

The Nano R4 board features built-in UART (Universal Asynchronous Receiver-Transmitter) communication that allows your projects to communicate with other devices through serial data transmission. UART is implemented within the RA4M1 microcontroller and provides two separate hardware serial ports: one connected to the USB-C connector for computer communication, and another available on pins `D0` and `D1` for external device communication. This makes it perfect for projects that need to communicate with sensors, modules or other microcontrollers while maintaining USB connectivity for debugging.

UART is particularly useful when your project needs to communicate with devices that require simple, reliable serial communication, rather than the more complex protocols like SPI or I²C. While SPI excels at high-speed communication and I²C is ideal for multiple device networks, UART provides straightforward point-to-point communication that works well with GPS modules, Bluetooth® modules, Wi-Fi® modules and other serial devices. UART communication is asynchronous, meaning it doesn't require a shared clock signal, making it robust over longer distances.

The Nano R4's UART interface offers the following technical specifications:

Parameter	Value	Notes
Baud Rates	300 to 2000000	Common: 9600, 115200
Data Bits	8-bit	Standard data width
Communication	Full-duplex	Simultaneous TX/RX
Hardware Ports	2	USB Serial + Serial1
UART Pins	<code>D0</code> , <code>D1</code>	RX, TX respectively
Operating Voltage	+5 VDC	TTL logic levels
Flow Control	Software	XON/XOFF supported

The Nano R4 board uses the following pins for UART communication:

Arduino Pin	Microcontroller Pin	UART Function	Description
<code>D0</code>	<code>P104</code>	RX	Receive Data
<code>D1</code>	<code>P105</code>	TX	Transmit Data

You can communicate via UART using the built-in `Serial` and `Serial1` objects. The `Serial` object is connected to the USB-C port for computer communication, while `Serial1`

The following example demonstrates basic UART communication patterns:

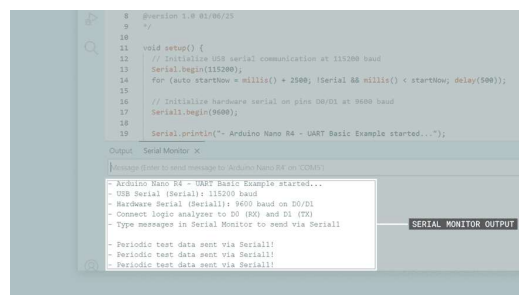
```

1  /**
2  UART Basic Example for the Arduino Nano R4
3  Name: nano_r4_uart_basic.ino
4  Purpose: This sketch demonstrates basic UART
5  using both USB Serial and hardware Serial1
6
7  @author Arduino Product Experience Team
8  @version 1.0 01/06/25
9  */
10
11 void setup() {
12   // Initialize USB serial communication at
13   Serial.begin(115200);
14   for (auto startNow = millis() + 2500; !Serial.available(); startNow = millis() + 2500);
15
16   // Initialize hardware serial on pins D0,
17   Serial1.begin(9600);
18
19   Serial.println("- Arduino Nano R4 - UART
20   Serial.println("- USB Serial (Serial): 115200 baud
21   Serial.println("- Hardware Serial (Serial1): 9600 baud
22   Serial.println("- Connect logic analyzer
23   Serial.println("- Type messages in Serial Monitor
24   Serial.println();
25 }
26
27 void loop() {
28   // Check for data from USB Serial (computer)

```

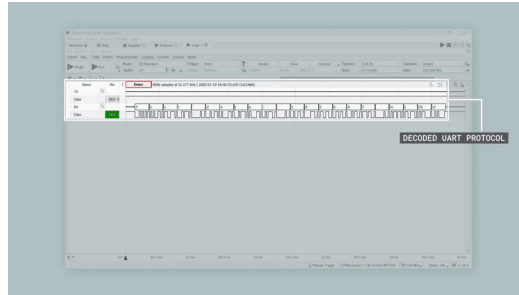
To test this example, no external UART devices are required, the code will demonstrate UART communication patterns that can be observed with a logic analyzer. You can type messages in the Arduino IDE's Serial Monitor to see them transmitted via `Serial1` on pins `D0` / `D1`.

You can open the Arduino IDE's Serial Monitor (Tools > Serial Monitor) to interact with the USB serial port and observe the communication patterns. Connect a logic analyzer to pins `D0` (RX) and `D1` (TX) to observe the actual UART protocol signals.



Arduino IDE Serial Monitor output for the UART

The image below shows how the UART communication from our example appears in the Digilent Waveforms logic analyzer software, with the decoded protocol showing the transmitted data bytes and timing.



Digilent Waveforms logic analyzer output for UART

The Nano R4 board provides two distinct UART communication channels, giving you the flexibility to handle multiple communication tasks simultaneously. The first channel is the USB Serial (`Serial`), which is your primary interface for programming and debugging. This channel offers several key features:

- Connected to the onboard USB-C connector
- Used for programming and debugging
- Typically runs at 115200 baud
- Automatic baud rate detection
- No external connections required

The second channel is the Hardware Serial (`Serial1`), which is dedicated to external device communication. This channel provides robust connectivity for your project peripherals:

- Connected to pins `D0` (RX) and `D1` (TX)
- Used for external device communication
- Configurable baud rate (300 to 2000000)
- TTL voltage levels (0 VDC/+5 VDC)
- Requires external device connection

Here is a practical example of how to use both UART channels simultaneously, such as when connecting a GPS module:



```
1 // Example: Connecting a GPS module via Serial
2 void setup() {
3   Serial.begin(115200); // USB debugging
4   Serial1.begin(9600); // GPS module communication
5
6   Serial.println("GPS module communication started");
7 }
8
9 void loop() {
10  // Forward GPS data to USB Serial for monitoring
11  if (Serial1.available()) {
12    String gpsData = Serial1.readString();
13    Serial.print("GPS: ");
14    Serial.print(gpsData);
15  }
16
17  // Send commands to GPS module from USB Serial
18  if (Serial.available()) {
19    String command = Serial.readString();
20    Serial1.print(command);
21    Serial.println("Command sent to GPS");
22  }
23 }
```

When working with UART on the Nano R4, there are several key points to keep in mind for successful implementation:

The dual UART design is different from the UNO R3 board that shared one UART between USB and pins D0/D1. This separation allows simultaneous USB debugging and external device communication.

Always ensure that both devices use the same baud rate, data bits (typically 8), and stop bits (typically 1) for successful communication.

Keep in mind that UART communication uses TTL voltage levels (0 VDC for logic **LOW**, +5 VDC for logic **HIGH**). If you need to communicate over longer distances or with RS-232 devices, you'll need a level converter.

When connecting external devices, remember that TX connects to RX and RX connects to TX (crossover connection).

The Nano R4's UART ports are full-duplex, meaning they can send and receive data simultaneously, making them perfect for interactive communication with modules like GPS, Bluetooth®, Wi-Fi®, or other microcontrollers.

SPI Communication

The Nano R4 board features built-in SPI (Serial Peripheral Interface) communication that allows your projects to communicate with external devices like sensors, displays, memory cards and other microcontrollers. SPI is implemented within the RA4M1 microcontroller and uses four dedicated pins to provide high-speed synchronous serial

SPI is particularly useful when your project needs to communicate with external components at high speeds, rather than using slower protocols. While I²C is perfect for simple sensor communication and UART for basic serial data exchange, SPI excels at high-speed communication with devices like SD cards, TFT displays, wireless modules or external memory chips. SPI can achieve much faster data rates than I²C and can handle multiple devices on the same bus through individual chip select lines.

The Nano R4's SPI interface offers the following technical specifications:

Parameter	Value	Notes
Clock Speed	Up to 24 MHz	Maximum SPI frequency
Data Transfer	8-bit	Standard data width
Communication	Full-duplex	Simultaneous send/receive
SPI Pins	D10 - D13	CS, MOSI, MISO, SCK
Multiple Devices	Supported	Via different CS pins
Operating Voltage	+5 VDC	Same as board
Protocol Support	Mode 0,1,2,3	All SPI modes available

The Nano R4 board uses the following pins for SPI communication:

Arduino Pin	Microcontroller Pin	SPI Function	Description
D10	P103	CS	Chip Select
D11	P101	MOSI	Master Out, Slave In
D12	P100	MISO	Master In, Slave Out
D13	P102	SCK	Serial Clock

You can communicate via SPI using the dedicated `SPI.h` library, which is included in the Arduino UNO R4 Boards core. The library provides simple functions to initialize the bus, send and receive data and manage multiple devices.

The following example demonstrates how to use SPI communication to send and receive data:

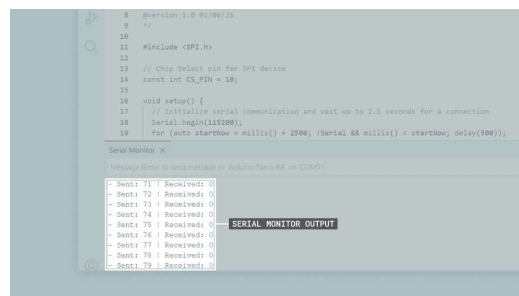
```

1  /**
2  SPI Basic Example for the Arduino Nano R4 F
3  Name: nano_r4_spi_basic.ino
4  Purpose: This sketch demonstrates how to us
5  to send and receive data.
6
7  @author Arduino Product Experience Team
8  @version 1.0 01/06/25
9  */
10
11 #include <SPI.h>
12
13 // Chip Select pin for SPI device
14 const int CS_PIN = 10;
15
16 void setup() {
17   // Initialize serial communication and wa
18   Serial.begin(115200);
19   for (auto startNow = millis() + 2500; !Se
20
21   Serial.println("- Arduino Nano R4 - SPI F
22
23   // Set CS pin as output and set it HIGH (
24   pinMode(CS_PIN, OUTPUT);
25   digitalWrite(CS_PIN, HIGH);
26
27   // Initialize SPI communication
28   SPI.begin();

```

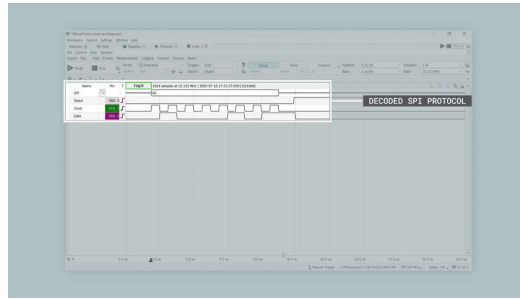
To test this example, no external SPI device is required. The code will demonstrate SPI communication patterns, though without a connected device, the received data will typically be `0xFF`.

You can open the Arduino IDE's Serial Monitor (Tools > Serial Monitor) to see the SPI communication in action. The example sketch shows how to properly select devices, send data and handle responses.



Arduino IDE Serial Monitor output for the SPI example sketch

The image below shows how the SPI communication from our example appears in the Digilent Waveforms logic analyzer software, with the decoded protocol showing the chip select, clock and data signals being transmitted.



Digilent Waveforms logic analyzer output for SPI

For connecting multiple SPI devices, you can use different digital pins as additional Chip Select (`CS`) lines while sharing the `MOSI`, `MISO` and `SCK` pins:

```
1 // Multiple device example
2 const int DEVICE1_CS = 10; // First SPI dev
3 const int DEVICE2_CS = 9;  // Second SPI de
4 const int DEVICE3_CS = 8;  // Third SPI dev
5
6 void setup() {
7   SPI.begin();
8
9   // Configure all CS pins
10  pinMode(DEVICE1_CS, OUTPUT);
11  pinMode(DEVICE2_CS, OUTPUT);
12  pinMode(DEVICE3_CS, OUTPUT);
13
14  // Set all CS pins HIGH (inactive)
15  digitalWrite(DEVICE1_CS, HIGH);
16  digitalWrite(DEVICE2_CS, HIGH);
17  digitalWrite(DEVICE3_CS, HIGH);
18 }
```

When working with SPI on the Nano R4, there are several key points to keep in mind for successful implementation:

The SPI protocol requires careful attention to timing and device selection. Always ensure that only one device is selected (`CS LOW`) at a time, and remember to deselect devices (`CS HIGH`) after communication to avoid conflicts.

Different SPI devices may require different clock speeds and modes, so check your device's datasheet for the correct `SPISettings()` parameters.

Keep in mind that SPI is a synchronous protocol, meaning that data is transferred in both directions simultaneously with each clock pulse. Even if you only need to send data, you'll still receive data back, and vice versa.

The Nano R4 board can communicate with multiple SPI devices by using different Chip Select (`CS`) pins, making it perfect for complex projects that need to interface with various sensors, displays and storage devices.

I2C Communication

The Nano R4 board features built-in I²C (Inter-Integrated Circuit) communication that allows your projects to communicate with multiple devices using just two wires. I²C is implemented within the RA4M1 microcontroller and uses two dedicated pins to provide reliable serial communication with sensors, displays, memory modules and other microcontrollers. This makes it perfect for projects that need to connect several devices without using many pins.

CRITICAL WARNING: The Nano R4's I²C pins (A4/A5) operate at +5 VDC logic levels WITHOUT built-in level shifting.  Directly connecting +3.3 VDC I²C devices to these pins may damage them permanently. This is different from many other Nano family boards that operate at +3.3 VDC.

Understanding the voltage differences between the Nano R4's I²C interfaces is important for protecting your components. The board offers two different I²C connection methods, each with distinct voltage characteristics. The following table summarizes these critical differences:

Interface	Operating Voltage	Level Shifting	Safe for +3.3 VDC Devices
Pins A4/A5	+5 VDC	None	 No (Requires level shifter)
Qwiic Connector	+3.3 VDC	Built-in	 Yes (direct connection)

When you need to connect +3.3 VDC I²C devices to your Nano R4 board, you have three main options to ensure safe and reliable operation:

- Use the Qwiic connector **: This is the simplest solution as it provides automatic level translation and +3.3 VDC power supply for your devices
- Add an external level shifter:** Bi-directional I²C level translators like the [SparkFun Logic Level Converter \(BOB-12009\)](#) or the [Adafruit 4-Channel I2C-Safe Bi-directional Logic Level Converter \(BSS138\)](#) can safely convert between +5 VDC and +3.3 VDC logic levels
- Use +5 VDC-compatible sensors:** Select I²C devices specifically designed to operate at +5 VDC to avoid any compatibility issues

I²C is particularly useful when your project needs to communicate with multiple sensors and devices in a simple way, rather than using complex wiring. While SPI is excellent for high-speed communication and UART for basic serial data exchange, I²C excels at connecting many devices with minimal wiring. Multiple I²C devices can share the same two-wire bus, each with its own unique address, making it ideal for sensor networks, display modules and expandable systems.

The Nano R4's I²C interface offers the following technical specifications:

Parameter	Value	Notes
Clock Speed	Up to 400 kHz	Standard/Fast mode
Data Transfer	8-bit	Standard data width
Communication	Half-duplex	One direction at a time
I ² C Pins	A4 , A5	SDA, SCL respectively
Device Addressing	7-bit/10-bit	Up to 127 unique addresses
Operating Voltage	+5 VDC	Same as board
Pull-up Resistors	External required	No internal pull-ups

The Nano R4 uses the following pins for I²C communication:

Arduino Pin	Microcontroller Pin	I ² C Function	Description
A4	P004	SDA	Serial Data Line
A5	P010	SCL	Serial Clock Line

You can communicate via I²C using the dedicated `Wire.h` library, which is included in the Arduino UNO R4 Boards core. The library provides simple functions to initialize the bus, send and receive data and manage multiple devices.

 **Important note:** The Nano R4 board has two I²C interfaces. Use `Wire` for the standard I²C interface on pins `A4/A5` (+5 VDC logic), and `Wire1` for the Qwiic connector (+3.3 VDC with built-in level shifting). This dual-interface design allows you to separate +5 VDC and +3.3 VDC I²C devices on different buses if needed.

The following example demonstrates basic I²C communication patterns:

```

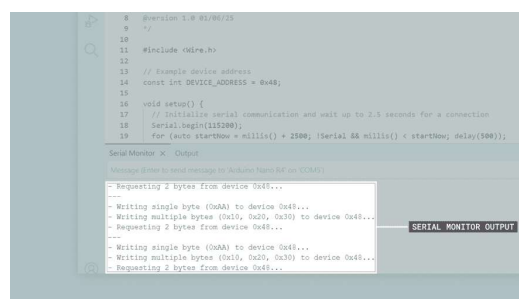
1  /**
2  I2C Basic Example for the Arduino Nano R4 F
3  Name: nano_r4_i2c_basic.ino
4  Purpose: This sketch demonstrates basic I2C
5  patterns for protocol analysis.
6
7  @author Arduino Product Experience Team
8  @version 1.0 01/06/25
9  */
10
11 #include <Wire.h>
12
13 // Example device address
14 const int DEVICE_ADDRESS = 0x48;
15
16 void setup() {
17   // Initialize serial communication and wait
18   Serial.begin(115200);
19   for (auto startNow = millis() + 2500; !Se
20
21   Serial.println("- Arduino Nano R4 - I2C f
22
23   // Initialize I2C communication as master
24   Wire.begin();
25
26   Serial.println("- I2C initialized succes
27   Serial.println("- Connect protocol analy
28   Serial.println("- Starting I2C communicat

```

To test this example, no external I²C devices are required.

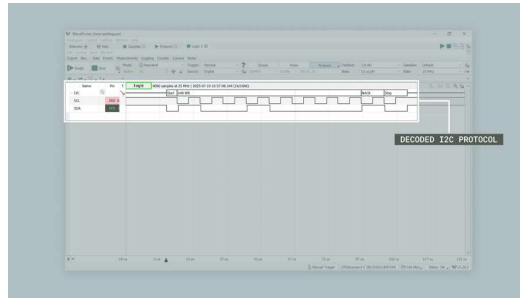
i The code will generate I²C communication patterns that can be analyzed with a protocol analyzer. Without devices connected, read operations will typically return `0xFF`.

You can open the Arduino IDE's Serial Monitor (Tools > Serial Monitor) to see the I²C operations being performed. Connect a protocol analyzer to pins `A4` (SDA) and `A5` (SCL) to observe the actual I²C protocol signals.



Arduino IDE Serial Monitor output for the I²C example sketch

The image below shows how the I²C communication from our example appears in the Digilent Waveforms logic analyzer software, with the decoded protocol showing the device address and data bytes being transmitted.



Arduino IDE Serial Monitor output for the I²C example sketch

The I²C protocol requires pull-up resistors on both SDA and SCL lines. **The Nano R4 board does not have internal pull-ups on A4 and A5 to avoid interference with their analog input functionality, so external 4.7kΩ pull-up resistors to +5 VDC are required for proper I²C operation.**

One of the main advantages of I²C is the ability to connect multiple devices to the same bus. Here is how to connect multiple I²C devices:

```

1 // Example: Communicating with multiple I2C
2 void communicateWithMultipleDevices() {
3   // Device addresses (examples)
4   const int SENSOR_ADDRESS = 0x48;    // Temperature sensor
5   const int DISPLAY_ADDRESS = 0x3C;    // OLED display
6   const int EEPROM_ADDRESS = 0x50;    // EEPROM
7
8   // Read from temperature sensor
9   Wire.beginTransmission(SENSOR_ADDRESS);
10  Wire.write(0x00); // Register to read
11  Wire.endTransmission();
12
13  Wire.requestFrom(SENSOR_ADDRESS, 2);
14  if (Wire.available() >= 2) {
15    int tempHigh = Wire.read();
16    int tempLow = Wire.read();
17    Serial.print("Temperature: ");
18    Serial.println((tempHigh << 8) | tempLow);
19  }
20
21  // Send data to display
22  Wire.beginTransmission(DISPLAY_ADDRESS);
23  Wire.write(0x40); // Data mode
24  Wire.write(0xFF); // Sample data
25  Wire.endTransmission();
26
27  // Write to EEPROM
28  Wire.beginTransmission(EEPROM_ADDRESS);
29  Wire.write(0x00); // Address
30  Wire.write(0xFF); // Data
31  Wire.endTransmission();
32
33  // Read from EEPROM
34  Wire.requestFrom(EEPROM_ADDRESS, 2);
35  if (Wire.available() >= 2) {
36    int eepromData = Wire.read();
37    Serial.println(eepromData);
38  }
39
40  // Delay between operations
41  delay(1000);
42
43  // Repeat the process for the display and EEPROM
44  communicateWithMultipleDevices();
45
46  // End of the function
47  }

```

When working with I²C on the Nano R4 board, there are several key points to keep in mind for successful

Each I²C device must have a unique address on the bus, so check device datasheets to avoid address conflicts.

Keep in mind that I²C is a half-duplex protocol, meaning data flows in only one direction at a time. The master device (your Nano R4 board) controls the clock line and initiates all communication.

When connecting multiple devices, simply connect all SDA pins together and all SCL pins together, along with power and ground connections.

The Nano R4 board can communicate with up to 127 different I²C devices on the same bus, making it perfect for complex sensor networks and expandable systems.

CAN Communication

The Nano R4 board features built-in CAN (Controller Area Network) communication that allows your projects to communicate with automotive systems, industrial devices, and other CAN-enabled equipment. CAN is implemented within the RA4M1 microcontroller and uses two dedicated pins to provide robust, real-time communication over longer distances with multiple devices. This makes it perfect for automotive applications, industrial automation and distributed control systems that require reliable communication in noisy environments.

CAN is particularly useful when your project needs to communicate in industrial or automotive environments where reliability and noise immunity are critical, rather than simple point-to-point communication. While UART excels at simple serial communication and SPI provides high-speed data transfer, CAN offers multi-node networking with built-in error detection, collision handling and message prioritization. CAN communication is differential signaling, making it highly resistant to electromagnetic interference and suitable for harsh environments.

The Nano R4's CAN interface offers the following technical specifications:

Parameter	Value	Notes
Baud Rates	Up to 1 Mbps	Configurable speed
Data Frames	0-8 bytes	Standard CAN frame
Communication	Multi-master	Multiple nodes support
CAN Pins	D4 , D5	TX, RX respectively
Message IDs	11-bit/29-bit	Standard/Extended format
Operating Voltage	+5 VDC	Requires CAN transceiver
External Hardware	Required	CAN transceiver needed

The Nano R4 uses the following pins for CAN communication:

Arduino Pin	Microcontroller Pin	CAN Function	Description
D4	P109	CAN_TX	CAN Transmit
D5	P110	CAN_RX	CAN Receive

Important note: The Nano R4's CAN interface provides logic-level signals only. You must use an external CAN transceiver (such as SN65HVD230 from Texas Instruments® or the MCP2561 from Microchip®) to convert these signals to the differential CAN bus levels (CANH / CANL) required for actual CAN communication.

You can communicate via CAN using the built-in `Arduino_CAN` library included in the Arduino UNO R4 Boards core. The library provides functions to initialize the bus, send and receive CAN frames and handle message filtering.

The following example demonstrates basic CAN communication patterns:

```
1 /**
2  CAN Basic Example for the Arduino Nano R4
3  Name: nano_r4_can_basic.ino
4  Purpose: This sketch demonstrates basic CAN
5  for automotive and industrial applications
6
7  @author Arduino Product Experience Team
8  @version 1.0 01/06/25
9  */
10
11 #include <Arduino_CAN.h>
12
13 void setup() {
14   // Initialize serial communication and wait
15   Serial.begin(115200);
16   for (auto startNow = millis() + 2500; !Serial
17
18   Serial.println("- Arduino Nano R4 - CAN I
19
20   // Initialize CAN communication at 500k b
21   if (!CAN.begin(CanBitRate::BR_500k)) {
22     Serial.println("- Error: Failed to init
23     while (1) delay(10);
24   }
25
26   Serial.println("- CAN bus initialized at
27   Serial.println("- Connect CAN transceiver
28   Serial.println("- Sending periodic CAN me
```

To test this example, you need an external CAN transceiver connected to pins `D4` and `D5`. The code will



demonstrate CAN communication patterns that can be observed with a CAN analyzer. Without a CAN transceiver and CAN network, the messages will not be transmitted on the actual CAN bus.

Since the Nano R4's CAN interface provides only logic-level signals, you'll need an external CAN transceiver to convert these signals to the differential voltage levels required by CAN networks. The transceiver acts as a bridge between your Nano R4 board and the actual CAN bus, handling the electrical requirements and providing isolation.

For prototyping with the Nano R4 board, you need a +5 VDC-compatible transceiver like the [MCP2561 from Microchip](#), which matches the board's +5 VDC logic levels. While the [SN65HVD230 from Texas Instruments](#) is popular and widely used, it operates at +3.3 VDC and may require level shifting when used with the +5 VDC Nano R4.

When working with CAN on the Nano R4, there are several key points to keep in mind for successful implementation:

- Always ensure proper termination with 120 Ω resistors at both ends of the CAN bus to prevent signal reflections.

- Keep in mind that CAN uses message-based communication with unique identifiers rather than device addresses. Lower ID numbers have higher priority, so critical messages should use lower IDs.

- The CAN protocol provides automatic error detection, retransmission and collision resolution, making it extremely reliable for safety-critical applications.

- The Nano R4 supports both standard (11-bit) and extended (29-bit) CAN identifiers, allowing compatibility with most automotive and industrial CAN networks.

HID (Human Interface Device) Communication

The Nano R4 board features built-in HID (Human Interface Device) communication that allows your projects to emulate input devices like keyboards and mice. HID is implemented within the RA4M1 microcontroller and uses the USB-C connection to communicate directly with computers as standard input devices. This makes it perfect for automation projects, assistive technologies, gaming controllers and interactive installations that need to control computers or send keystrokes and mouse movements.

HID is particularly useful when your project needs to interact directly with computer software, rather than just sending

interpret data, HID devices are recognized automatically by any operating system as standard keyboards or mice. This allows your Nano R4 to trigger keyboard shortcuts, type text, move the mouse cursor, or click buttons, making it ideal for presentation controllers, accessibility devices and automation systems.

The Nano R4's HID interface offers the following technical specifications:

Parameter	Value	Notes
Device Types	Keyboard & Mouse	Standard HID classes
Connection	USB-C	Native USB connection
Compatibility	Cross-platform	Windows, macOS, Linux
Key Modifiers	All standard	Ctrl, Alt, Shift, GUI/Cmd
Mouse Buttons	3 buttons	Left, Right, Middle
Mouse Movement	Relative/Delta	Smooth cursor movement
Scroll Wheel	Supported	Vertical scroll only
Multiple Devices	Supported	Keyboard + Mouse together

The Nano R4 uses the USB-C connector for HID communication:

Connection	Function	Description
USB-C	HID	Keyboard and Mouse emulation

Important note: When HID functionality is active, the Nano R4 board will be recognized by your computer as an  input device. Be careful with your code to avoid sending unintended keystrokes or mouse movements that could interfere with normal computer operation.

You can use HID communication through the dedicated `Keyboard.h` and `Mouse.h` libraries, which are included in the Arduino UNO R4 Boards core. These libraries provide simple functions to send keystrokes, key combinations, mouse movements and clicks.

The following example demonstrates basic keyboard emulation capabilities:

```

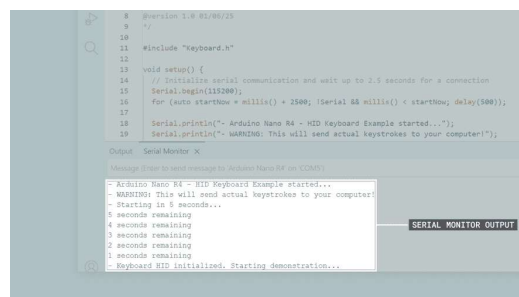
1  /**
2  HID Keyboard Example for the Arduino Nano R4
3  Name: nano_r4_hid_keyboard.ino
4  Purpose: This sketch demonstrates how to use an
5  USB keyboard to send keystrokes and key codes
6
7  @author Arduino Product Experience Team
8  @version 1.0 01/06/25
9  */
10
11 #include "Keyboard.h"
12
13 void setup() {
14   // Initialize serial communication and wait
15   Serial.begin(115200);
16   for (auto startNow = millis() + 2500; !Serial.available(); startNow = millis() + 2500) {}
17
18   Serial.println("- Arduino Nano R4 - HID Keyboard Example started...");
19   Serial.println("- WARNING: This will send actual keystrokes to your computer!");
20
21   // Wait 5 seconds before starting HID to
22   Serial.println("- Starting in 5 seconds.");
23   for (int i = 5; i > 0; i--) {
24     Serial.print(i);
25     Serial.println(" seconds remaining");
26     delay(1000);
27   }
28
29   Keyboard.begin();
30   Keyboard.print("Arduino Nano R4 - HID Keyboard Example started...");
31   Keyboard.println();
32   Keyboard.print("- WARNING: This will send actual keystrokes to your computer!");
33   Keyboard.println();
34   Keyboard.print("Starting in 5 seconds...");
35   Keyboard.println();
36   for (int i = 5; i > 0; i--) {
37     Keyboard.print(i);
38     Keyboard.println(" seconds remaining");
39     delay(1000);
40   }
41   Keyboard.print("Keyboard initialized. Starting demonstration...");
42   Keyboard.println();
43 }

```

To test this example, no external components are needed.

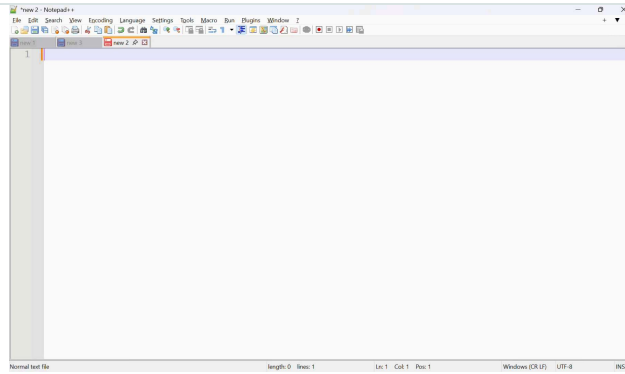
- i** The code will wait 5 seconds before activating HID functionality, then automatically demonstrate various keyboard functions.

You can open the Arduino IDE's Serial Monitor (Tools > Serial Monitor) to see the status messages.



Arduino IDE Serial Monitor output for the keyboard emulation example sketch

After 5 seconds, the Nano R4 board will type text directly into whatever application has focus on your computer.



The following example demonstrates mouse emulation capabilities:

```
1  /**
2  HID Mouse Example for the Arduino Nano R4
3  Name: nano_r4_hid_mouse.ino
4  Purpose: This sketch demonstrates how to use
5  USB mouse to control cursor movement and click
6
7  @author Arduino Product Experience Team
8  @version 1.0 01/06/25
9  */
10
11 #include "Mouse.h"
12
13 void setup() {
14   // Initialize serial communication and wait
15   Serial.begin(115200);
16   for (auto startNow = millis() + 2500; !Serial.available(); startNow++) {}
17
18   Serial.println("- Arduino Nano R4 - HID Mouse Example");
19   Serial.println("- WARNING: This will control your cursor!");
20
21   // Wait 5 seconds before starting HID
22   Serial.println("- Starting in 5 seconds.");
23   for (int i = 5; i > 0; i--) {
24     Serial.print(i);
25     Serial.println(" seconds remaining");
26     delay(1000);
27   }
28 }
```

To test this example, no external components are needed.



The code will automatically demonstrate mouse movement, clicking and scrolling functions.

You can open the Arduino IDE's Serial Monitor (Tools > Serial Monitor) to see the status messages.

```

17 // (auto startNow = millis()) + 2500; (Serial BA millis()) < startNow; delay(1000);
18 Serial.println("- Arduino Nano R4 - HID Mouse Example started...");
19 Serial.println("- WARNING: This will control your actual mouse cursor!");
20
21 // Wait 5 seconds before starting HID
22 Serial.println("- Starting in 5 seconds...");
23 for (int i = 5; i > 0; i--) {
24   Serial.println(i);
25   Serial.println("seconds remaining");
26   delay(1000);
27 }

```

Serial Monitor X

Message (Click to send message to Arduino Nano R4 on COM4)

```

- Arduino Nano R4 - HID Mouse Example started...
- WARNING: This will control your actual mouse cursor!
- Starting in 5 seconds...
5 seconds remaining
4 seconds remaining
3 seconds remaining
2 seconds remaining
1 seconds remaining
- Mouse HID initialized. Starting demonstration...

```

SERIAL MONITOR OUTPUT

Arduino IDE Serial Monitor output for the mouse emulation example sketch

The mouse cursor will move and click according to the button presses.

```

1 /*
2  * HID Mouse Example for the Arduino Nano R4 Board
3  * Name: nano_r4_hid_mouse.ino
4  * Purpose: This sketch demonstrates how to use the Nano R4 as a
5  * USB mouse to control cursor movement and clicking.
6  *
7  * Author: Arduino Product Experience Team
8  * @version 1.0 01/06/23
9  */
10
11 #include "Mouse.h"
12
13 void setup() {
14   // Initialize serial communication and wait up to 2.5 seconds for a connection
15   Serial.begin(115200);
16   for (auto startNow = millis() + 2500; (Serial BA millis()) < startNow; delay(1000);
17     Serial.println("- Arduino Nano R4 - HID Mouse Example started...");
18     Serial.println("- WARNING: This will control your actual mouse cursor!");
19   }

```

Serial Monitor X

Message (Click to send message to Arduino Nano R4 on COM4)

```

- Arduino Nano R4 - HID Mouse Example started...
- WARNING: This will control your actual mouse cursor!
- Starting in 5 seconds...
5 seconds remaining
4 seconds remaining
3 seconds remaining
2 seconds remaining
1 seconds remaining

```

10:11, Col 10 - Arduino Nano R4 on COM4

When working with HID on the Nano R4, there are several key points to keep in mind for successful implementation:

HID functionality makes your Nano R4 board appear as a real keyboard and mouse to your computer, so always include safeguards and delays in your code to prevent unintended actions that could interfere with normal computer operation.

Always include a delay or warning period before activating HID functionality to give yourself time to prepare or stop the program if needed.

Keep in mind that HID devices are automatically recognized by all modern operating systems without requiring special drivers, making your projects instantly compatible with Windows, macOS and Linux.

For debugging HID projects, use `Serial.println()` statements to track what your code is doing, since you cannot rely on the Arduino IDE's Serial Monitor once HID actions start interfering with your computer.

The Nano R4 can function as both a keyboard and mouse simultaneously, allowing for complex automation sequences that combine typing, shortcuts and mouse control.

Remember that different operating systems may have slightly different keyboard layouts and shortcuts, so test your HID projects on your target platform to ensure compatibility.

The Nano R4 features external interrupt capability through the RA4M1 microcontroller's ICU (Interrupt Control Unit). Interrupts allow your Nano R4 board to immediately respond to pin state changes by temporarily pausing the main program to execute an Interrupt Service Routine (ISR), then returning to where it left off. This makes interrupts essential for time-critical applications like button detection, encoder reading, and sensor monitoring.

Interrupt Specifications

The Nano R4 board's external interrupt capabilities offer the following technical specifications:

Parameter	Value	Notes
Hardware Channels	9	Channels 0-7, 9 available
Interrupt-capable Pins	13 pins	7 digital + 6 analog pins
Trigger Modes	4	RISING, FALLING, CHANGE, and LOW
Response Time	< 1 μ s	Typical interrupt latency

Interrupt-Capable Pins

The following pins support external interrupts on the Nano R4 board:

Arduino Pin	Interrupt Channel	Primary Function	Notes
D0	Channel 6	Digital I/O	Shares channel with A1
D1	Channel 5	Digital I/O	Dedicated channel
D2	Channel 0	Digital I/O	Recommended for interrupts
D3	Channel 1	Digital I/O, PWM	Shares channel with A4
D8	Channel 9	Digital I/O	Dedicated channel
D12	Channel 3	Digital I/O, MISO	Shares channel with A6
D13	Channel 4	Digital I/O, SCK	Dedicated channel
A1	Channel 6	Analog, OPAMP+	Shares channel with D0
A2	Channel 7	Analog, OPAMP-	Dedicated channel

Arduino Pin	Interrupt Channel	Primary Function	Notes
A4	Channel 1	Analog, I ² C SDA	Shares channel with D3
A5	Channel 2	Analog, I ² C SCL	Shares channel with A3
A6	Channel 3	Analog input	Shares channel with D12

Important note: Pins sharing the same interrupt channel cannot be used for interrupts simultaneously. For example, D3 and A4 both use channel 1, so only one can be configured for interrupt functionality at a time.

Interrupt Trigger Modes

The Nano R4 board supports four interrupt trigger modes:

Mode	Trigger Condition	Typical Use Cases
RISING	Pin transitions from LOW to HIGH	Button press detection (pull-down)
FALLING	Pin transitions from HIGH to LOW	Button press detection (pull-up)
CHANGE	Pin changes state (either direction)	Encoder reading, pulse counting
LOW	Pin remains at LOW level	Level-triggered events

Important note: The HIGH trigger mode is not supported by the hardware. If specified, it will behave as RISING mode (detecting only the LOW-to-HIGH transition, not the continuous HIGH state). For continuous HIGH level detection, use polling with digitalRead() instead.

You can attach interrupts using the dedicated Arduino functions:

```
1 attachInterrupt(digitalPinToInterrupt(pin), I
2 detachInterrupt(digitalPinToInterrupt(pin));
```

The following example demonstrates basic interrupt usage:

```

1  /**
2  External Interrupt Example for the Arduino
3  Name: nano_r4_interrupt_example.ino
4  Purpose: This sketch demonstrates how to use
5  to detect button presses.
6
7  @author Arduino Product Experience Team
8  @version 1.0 01/06/25
9  */
10
11 const int buttonPin = 2;
12 const int ledPin = LED_BUILTIN;
13
14 // Variables shared with ISR must be volatile
15 volatile bool buttonPressed = false;
16 volatile unsigned long pressCount = 0;
17
18 // Interrupt Service Routine - keep it short
19 void buttonISR() {
20   buttonPressed = true;
21   pressCount++;
22 }
23
24 void setup() {
25   // Initialize serial communication and wait
26   Serial.begin(115200);
27   for (auto startNow = millis() + 2500; !Serial.available(); startNow = millis()) {}
28

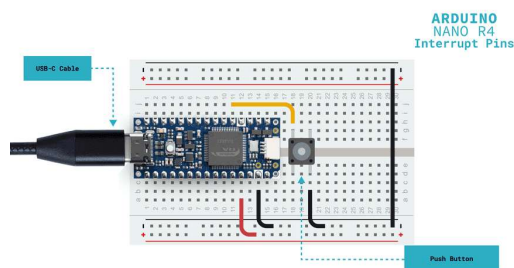
```

To test this example, connect a push button to the Nano R4 board as follows:

Connect one leg of a push button to pin **D2**

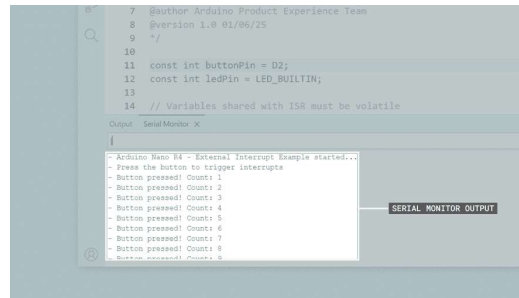
Connect the other leg of the push button to **GND**

No external components needed (using built-in LED and internal pull-up)



Interrupt pins test circuit on the Nano R4 board

Open the Arduino IDE's Serial Monitor (Tools > Serial Monitor) to see the interrupt count increase with each button press.



```
7 @author: Arduino Product Experience Team
8 @version 1.0 01/06/25
9 */
10
11 const int buttonPin = D2;
12 const int ledPin = LED_BUILTIN;
13
14 // Variables shared with ISR must be volatile
15
16 void setup() {
17   pinMode(buttonPin, INPUT_PULLUP);
18   pinMode(ledPin, OUTPUT);
19 }
20
21 void loop() {
22   digitalWrite(ledPin, HIGH);
23   delay(1000);
24   digitalWrite(ledPin, LOW);
25   delay(1000);
26 }
27
28 void buttonPressed() {
29   count++;
30   Serial.print("Button pressed! Count: ");
31   Serial.println(count);
32 }
```

Serial Monitor Output:

```
Arduino Nano R4 - External Interrupt Example started...
Press the button to trigger interrupts
Button pressed! Count: 1
Button pressed! Count: 2
Button pressed! Count: 3
Button pressed! Count: 4
Button pressed! Count: 5
Button pressed! Count: 6
Button pressed! Count: 7
Button pressed! Count: 8
```

Arduino IDE Serial Monitor output for the basic interrupt example sketch

When working with interrupts on the Nano R4 board, there are several key points to keep in mind for successful implementation:

Keep ISR functions short and fast: Avoid `delay()`, `Serial.print()`, or complex calculations inside ISRs as they block other interrupts.

Use volatile variables: Always declare variables shared between ISRs and main code as volatile to prevent compiler optimization issues.

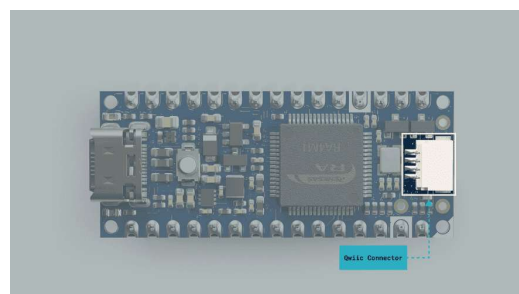
Manage channel conflicts: Verify that pins don't share the same interrupt channel when using multiple interrupts.

Consider debouncing: Mechanical switches may cause multiple triggers (add a 100nF capacitor or implement software debouncing).

Protect shared multi-byte variables: Disable interrupts temporarily when accessing them.

Qwiic Connector

The Nano R4 board features an onboard Qwiic connector that provides a simple, tool-free solution for connecting I²C devices. The Qwiic ecosystem, developed by SparkFun Electronics, has become an industry standard for rapid prototyping with I²C devices, allowing you to connect sensors, displays, and other peripherals without soldering or complex wiring. This makes it perfect for quickly building sensor networks, adding displays, or expanding your project's capabilities with minimal effort.



The Qwiic connector enhances the Nano R4's versatility by providing a dedicated interface for the extensive ecosystem of +3.3 VDC I²C modules. While the Nano R4's native +5 VDC operation ensures compatibility with classic Arduino shields and components, the Qwiic interface adds seamless integration with modern sensors and modules. The board includes built-in level translation between the +5 VDC microcontroller and the +3.3 VDC Qwiic bus, giving you the best of both worlds: full compatibility with traditional Arduino hardware on the main pins, plus instant access to the latest Qwiic-compatible devices.

Qwiic Specifications

The Nano R4's Qwiic connector offers the following specifications:

Parameter	Value	Notes
Connector Type	JST SH 4-pin 1mm	Industry standard Qwiic
Operating Voltage	+3.3 VDC	Powered by onboard regulator
I ² C Interface	<code>Wire1</code>	Secondary I ² C bus (not <code>Wire</code>)
Level Shifting	Built-in	Automatic +5 VDC to +3.3 VDC level translation
Pin Order	<code>GND</code> , <code>VCC</code> , <code>SDA</code> , <code>SCL</code>	Standard Qwiic pinout

The Qwiic connector is a small 4-pin connector with a 1.00 mm pitch. The mechanical details of the connector can be found in the connector's [datasheet](#).

The pin layout of the Qwiic connector is the following:

- `GND`: Ground
- `VCC`: +3.3 VDC power supply
- `SDA`: I²C data line
- `SCL`: I²C clock line

The `VCC` pin provides a stable +3.3 VDC output from the onboard regulator when the board is powered. The manufacturer part number of the Qwiic connector is **SM04B-SRSS-TB**, and its matching cable assemblies use the **SHR-04V-S-B** receptacle.

Understanding the Dual I²C Architecture

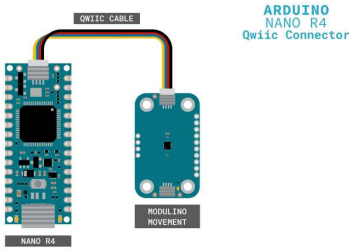
The Nano R4 implements a dual I²C bus architecture that separates +5 VDC and +3.3 VDC devices:

I ² C Bus	Arduino Object	Physical Connection	Voltage	Level Shifter	Use Case
Primary	<code>Wire</code>	Pins A4/A5	+5 VDC	None	+5V sensors, Arduino shields
Secondary	<code>Wire1</code>	Qwiic Connector	+3.3 VDC	Built-in	Modern sensors, Qwiic devices

Important note: The Qwiic connector uses `Wire1` object, not the standard `Wire` object. This is different from boards with a single I²C bus. Always use `Wire1.begin()` and related `Wire1` functions when communicating with Qwiic devices.

Connecting Qwiic Devices

Connecting Qwiic devices to your Nano R4 board is straightforward. Simply get a standard Qwiic cable (available in various lengths) and plug one end into the Nano R4's Qwiic connector and the other into your Qwiic device. Most Qwiic devices have two connectors, allowing you to daisy-chain multiple devices without any soldering or breadboarding. The polarized connectors prevent incorrect connections, making the setup process foolproof.



Nano R4 and Modulino Movement connected via Qwiic

The Qwiic system's key advantages include:

- Plug-and-play connectivity:** No breadboards, jumper wires, or soldering required
- Polarized connectors:** Prevents accidental reverse connections
- Daisy-chain capability:** Connect multiple devices in series
- Built-in pull-up resistors:** No external resistors needed
- Standard pinout:** Compatible across all Qwiic ecosystem devices

Working with Qwiic Devices

The following example demonstrates how to use the [Arduino Modulino Movement](#) sensor via the Qwiic connector:

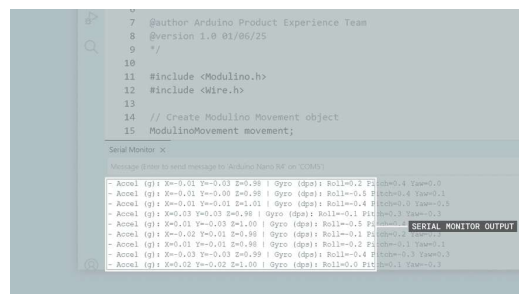
```

1  /**
2  Modulino Movement Example for the Arduino Nano R4
3  Name: nano_r4_modulino_movement.ino
4  Purpose: This sketch demonstrates reading the
5  Modulino Movement sensor via the Qwiic connector
6
7  @author Arduino Product Experience Team
8  @version 1.0 01/06/25
9  */
10
11 #include <Modulino.h>
12 #include <Wire.h>
13
14 // Create Modulino Movement object
15 ModulinoMovement movement;
16
17 // Variables for sensor data
18 float x, y, z;
19 float roll, pitch, yaw;
20
21 void setup() {
22   // Initialize serial communication and wait until ready
23   Serial.begin(115200);
24   for (auto startNow = millis() + 2500; !Serial; startNow = millis());
25
26   Serial.println("- Arduino Nano R4 - Modulino Movement");
27
28   // Initialize Wire1 for Qwiic connector

```

To test this example, you need an [Arduino Modulino Movement](#) connected to the Nano R4's Qwiic connector via a Qwiic cable. Install the Modulino library from the Arduino IDE Library Manager (Tools > Manage Libraries > Search "Modulino").

You can open the Arduino IDE's Serial Monitor (Tools > Serial Monitor) to see the real-time acceleration and gyroscope data. Try moving or rotating the sensor to see the values change.



This example demonstrates the key aspects of using Qwiic devices with the Nano R4:

Using `Wire1.begin()` instead of `Wire.begin()` for the Qwiic connector

Direct plug-and-play connection without additional wiring

Automatic +3.3 VDC power and level translation

Access to Arduino's ecosystem of Modulino sensors

Bootloader

The Nano R4 board features a built-in bootloader that enables sketch uploading directly from the Arduino IDE without requiring external programming hardware. The bootloader is a small program stored in a protected area of the RA4M1 microcontroller's flash memory that runs each time the board powers on or resets. It manages the critical communication between your computer and the board during sketch uploads, making the development process simple and accessible.

During normal operation, when you connect your Nano R4 to your computer and upload a sketch, the bootloader receives the new program data via USB and writes it to the appropriate memory location. This process happens automatically and transparently, allowing you to focus on developing your projects rather than managing low-level programming details.

Understanding the Bootloader Operation

When the Nano R4 board resets or powers on, the bootloader performs the following sequence:

- 1 **Initialization:** The bootloader initializes the USB communication and checks for incoming programming commands
- 2 **Wait Period:** It waits briefly for new sketch data from the Arduino IDE
- 3 **Program Execution:** If no new sketch is received, it jumps to your previously uploaded program

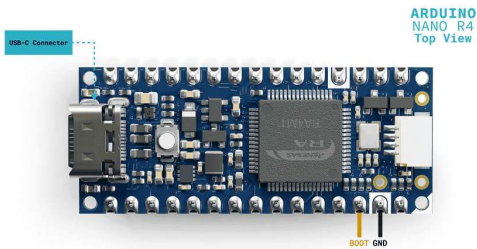
This automatic process ensures that your board is always ready to receive new sketches while maintaining quick startup times for your applications.

Bootloader Recovery

In certain situations, the bootloader may become corrupted or stop functioning properly. Common symptoms of

- The board is not recognized by the Arduino IDE
- Upload attempts fail with timeout errors
- The board appears as an unknown device in your computer's device manager
- The onboard LEDs behave abnormally during connection attempts

The Nano R4 includes a special hardware feature for bootloader recovery: the `BOOT` pin. This pin, when connected to ground (`GND`) during power-up, puts the board into a special programming mode that allows you to restore the bootloader using the Renesas Flash Programmer tool.



The Nano R4 BOOT pin (top view)

Important note: The `BOOT` pin is located on the bottom side of the board and is clearly labeled. This pin should only be used for bootloader recovery or advanced programming operations. During normal use, leave this pin unconnected.

Technical Specifications

The Nano R4's bootloader offers the following technical specifications:

Parameter	Value	Notes
Recovery Method	<code>BOOT</code> pin + <code>GND</code>	Hardware recovery mode activation
Bootloader File	<code>dfu_nano.hex</code>	Included with Arduino UNO R4 Boards core
Programming Tool	Renesas Flash Programmer	Official tool for bootloader recovery

Bootloader File Location

The bootloader file (`dfu_nano.hex`) can be obtained in two ways:

Option 1: Direct Download (recommended):

Download directly from the Arduino GitHub repository:

Option 2: From Arduino IDE Installation:

The bootloader file (`dfu_nano.hex`) is automatically installed with the Arduino UNO R4 Boards core. You can find it in the following locations:

Windows:

```
1 C:\Users\[YourUsername]\AppData\Local\Arduino
```

macOS:

```
1 ~/Library/Arduino15/packages/arduino/hardware
```

Linux:

```
1 ~/.arduino15/packages/arduino/hardware/renes
```



Replace `[YourUsername]` with your actual username and `[version]` with the installed core version number.

When to Consider Bootloader Recovery

Bootloader recovery should be considered when:

- Standard troubleshooting steps (different cables, USB ports, driver reinstallation) have failed

- The board was working previously but stopped being recognized after a failed upload

- Power was interrupted during a sketch upload

- You need to restore the board to factory programming state

- The board is not responding to the reset button normally

Bootloader Recovery Tutorial

For detailed step-by-step instructions on recovering and reflashing the bootloader on your Nano R4 board, please refer to our [Nano R4 Bootloader Recovery and Flashing Tutorial](#). This tutorial covers the following topics:

Installing and configuring the Renesas Flash Programmer tool

Step-by-step flashing process

Verifying successful bootloader restoration

Troubleshooting common issues

 **Important note:** Bootloader recovery should only be performed when necessary. Ensure stable power supply and follow the tutorial instructions carefully. The process requires only the Renesas Flash Programmer tool and the bootloader file.

Maintaining Bootloader Health

To minimize the risk of bootloader corruption and ensure reliable operation, we recommend the following:

Always allow uploads to complete: Never disconnect the USB cable or remove power during sketch uploads

Use quality USB cables: Poor quality or damaged cables can cause communication errors

Ensure stable power: Use a reliable power source when uploading sketches

Keep the Arduino IDE updated: Newer versions often include improvements to the upload process

Handle the board carefully: Avoid static discharge and physical damage to the board

The bootloader is a critical component that makes the Nano R4 board user-friendly and accessible. While it's designed to be robust and reliable, understanding its operation and recovery options ensures you can always restore your board to working condition if issues arise.

Support

If you encounter any issues or have questions while working with your Nano R4 board, we provide various support resources to help you find answers and solutions.

Help Center

Explore our Help Center, which offers a comprehensive collection of articles and guides for Nano family boards. The Help Center is designed to provide in-depth technical assistance and help you make the most of your device.

[Nano family help center page](#)

Forum

Forum is an excellent place to learn from others, discuss issues, and discover new ideas and projects related to the Nano R4.

[Nano category in the Arduino Forum](#)

Contact Us

Please get in touch with our support team if you need personalized assistance or have questions not covered by the help and support resources described before. We're happy to help you with any issues or inquiries about the Nano family boards.

[Contact us page](#)

Suggest changes

The content on docs.arduino.cc is facilitated through a public [GitHub repository](#). If you see anything wrong, you can edit this page [here](#).

Need support?

[Help Center](#)
[Ask the Arduino Forum](#)
[Discover Arduino Discord](#)

License

The Arduino documentation is licensed under the [Creative Commons Attribution-Share Alike 4.0](#) license.

Was this article helpful?

